

Apuntes CC1002 Introducción a la Programación

Francisco Gutiérrez Figueroa
Vanessa Peña Araya
Mauricio Quezada Veas
Benjamin Bustos
Romain Robbes

Estos apuntes del curso CC1002 Introducción a la Programación están basados en los libros *How to Design Programs*, MIT Press, de M. Felleisen et al., y *Objects First with Java - A Practical Introduction using BlueJ*, Fifth Edition, Prentice Hall, de David J. Barnes y Michael Kölling. La distribución de estos apuntes está limitada al cuerpo docente y a los alumnos del curso CC1002. ESTÁ PROHIBIDO COMPARTIR O REDISTRIBUIR ESTOS APUNTES FUERA DE LA COMUNIDAD DEL CURSO CC1002.

Índice

1	Expresiones y Tipos de Datos Básicos	1
1.1	¿Qué es un algoritmo?	1
1.2	¿Qué es un programa?	2
1.3	Tipos de datos básicos	2
1.3.1	Enteros (int)	2
1.3.2	Reales (float)	3
1.3.3	Texto (str)	3
1.4	Programas simples	3
1.4.1	Evaluación de expresiones	3
1.4.2	Variables	4
1.5	Errores	5
1.5.1	Errores de sintaxis	6
1.5.2	Errores de nombre	6
1.5.3	Errores de tipo	7
1.5.4	Errores de valor	7
1.5.5	Errores de indentación	7
1.5.6	Errores lógicos	7
2	Funciones	9
2.1	Variables y funciones	9
2.1.1	Definición de funciones	9
2.1.2	Indentación y subordinación de instrucciones	10
2.1.3	Alcance de una variable	11
2.2	Problemas	12
2.3	Un poco más sobre errores	13
2.3.1	Errores de ejecución	13
2.3.2	Errores de indentación	14
3	Receta de Diseño	15
3.1	Entender el propósito de la función	15
3.2	Dar ejemplos de uso de la función	17
3.3	Probar la función	17
3.4	Especificar el cuerpo de la función	18
4	Módulos y Programas	19
4.1	Descomponer un programa en funciones	20
4.2	Módulos	22
4.3	Programas interactivos	24

5	Expresiones y Funciones Condicionales	28
5.1	Valores booleanos	28
5.2	Funciones sobre booleanos	30
5.3	Condiciones	31
5.4	Bloques de código condicionales en Python	34
5.5	Diseñar funciones condicionales	35
5.5.1	Análisis de los datos y definición	35
5.5.2	Dar ejemplos de uso la función	36
5.5.3	El cuerpo de la función: diseñar condiciones	36
5.5.4	El cuerpo de la función: responder a cada condición	36
5.5.5	Simplificar condiciones	37
6	Recursión	39
6.1	Potencias, factoriales y sucesiones	39
6.2	Torres de Hanoi	41
6.3	Copo de nieve de Koch	43
6.4	Receta de diseño para la recursión	47
7	Testing y Depuración de Programas	49
7.1	Afirmaciones (<i>assertions</i>)	50
7.2	Testear con números reales	51
7.3	Ejemplo: cálculo de la raíz cuadrada	52
8	Datos Compuestos	54
8.1	Estructuras (<i>structs</i>)	54
8.2	Receta de diseño para estructuras	55
8.2.1	Diseñar estructuras	56
8.2.2	Plantilla	56
8.2.3	Cuerpo de la función	56
9	Estructuras de Datos Recursivas	58
9.1	Listas	58
9.2	Definición de datos para listas de largo arbitrario	60
9.3	Procesar listas de largo arbitrario	61
9.3.1	Receta de diseño para funciones con definiciones de datos recursivas	63
9.4	Funciones que producen listas	64
9.5	Listas que contienen estructuras	65
9.6	Modulo de listas	67
9.7	Otras definiciones de datos recursivas	68
9.8	Definiciones mutuamente recursivas	72
9.9	Árboles binarios	76
10	Abstracción Funcional	79
10.1	Similitudes en definiciones	79
10.2	Similitudes en definición de datos	84
10.3	Formalizar la abstracción a partir de ejemplos	85
10.3.1	Comparación	86
10.3.2	Abstracción	86
10.3.3	Test	87
10.3.4	Contrato	87
10.3.5	Formulando contratos generales	88
10.4	Otro ejemplo de función abstracta	89
10.5	Funciones anónimas	90

10.6	Resumen de funciones abstractas para listas	90
10.6.1	Función <code>filtro</code>	90
10.6.2	Función <code>mapa</code>	91
10.6.3	Función <code>fold</code>	91
11	Mutación	93
11.1	Memoria para funciones	93
11.2	Diseñar funciones con memoria	95
11.2.1	La necesidad de tener memoria	95
11.2.2	Memoria y variables de estado	96
11.2.3	Funciones que inicializan memoria	97
11.2.4	Funciones que cambian la memoria	98
11.3	Estructuras mutables	102
11.4	Diseñar funciones que modifican estructuras	103
11.4.1	¿Por qué mutar estructuras?	104
11.4.2	Receta de diseño estructural y con mutación	104
12	Estructuras Indexadas	108
12.1	Arreglos	108
12.2	Listas de Python	109
12.3	Iterar sobre estructuras indexadas	110
12.3.1	Instrucción <code>for</code>	110
12.3.2	Instrucción <code>while</code>	111
12.4	Strings como secuencias indexadas de caracteres	112
12.5	Diccionarios de Python	112
13	Depuración	114
13.1	¿Qué es la depuración?	114
13.2	El proceso de depuración	115
13.3	Depurar programas con el método científico	115
14	Objetos y Clases	117
14.1	Un ejemplo: automóviles	117
14.2	Crear e interactuar con objetos	117
14.3	Múltiples instancias de una clase y estado de los objetos	118
14.4	Ejemplo: libreta de direcciones	119
15	Definición de Clases	120
15.1	Clase	120
15.1.1	Campos	120
15.2	Constructor	120
15.3	Métodos	121
15.3.1	Métodos accesorios y mutadores	121
15.4	Receta de diseño de clases	122
16	Interacciones entre Objetos	127
16.1	Abstracción y modularidad con objetos	127
16.2	Diagramas de clases y objetos	128
16.3	Objetos que crean objetos	132
16.4	Llamadas a métodos	133
16.5	Testing de clases	135

17	Diseño de Clases	137
17.1	Introducción	137
17.2	Introducción al acoplamiento y la cohesión	139
17.3	Duplicación de código	139
17.4	Acoplamiento	142
17.4.1	Usar encapsulamiento para reducir el acoplamiento	143
17.5	Diseño basado en responsabilidades	146
17.5.1	Responsabilidades y acoplamiento	146
17.6	Cohesión	147
17.6.1	Cohesión de métodos	147
17.6.2	Cohesión de clases	148
17.6.3	Cohesión para lograr legibilidad	148
17.6.4	Cohesión para lograr reutilización	148
17.7	Refactoring	149
17.7.1	Refactoring y testing	149
18	Interfaces y Polimorfismo	151
18.1	¿Qué es una interfaz?	151
18.2	Ejemplo: animales	151
18.3	¿Qué es el polimorfismo?	152
18.4	Beneficios del polimorfismo	153
18.5	Otro ejemplo: puntos y líneas	154
18.6	Reuso de código con delegación y herencia	157
18.6.1	Delegación	157
18.6.2	Herencia	158

Capítulo 1

Expresiones y Tipos de Datos Básicos

Estas notas pretenden ser un complemento a las cátedras dictadas en el contexto del curso CC1002 Introducción a la Programación, obligatorio para alumnos de Primer Año del Plan Común de Ingeniería y Ciencias, dictado por la Facultad de Ciencias Físicas y Matemáticas de la Universidad de Chile.

El objetivo principal de este curso no es formar programadores, sino desarrollar en los alumnos una base común en razonamiento algorítmico y lógico, así como una capacidad de modelamiento y abstracción, necesarios para trabajar una habilidad general en la resolución de problemas. Estos problemas no necesariamente estarán acotados en el contexto de las Ciencias de la Computación, sino en el ámbito cotidiano y de la Ingeniería y Ciencias en general.

Los computadores son máquinas con un gran poder de cálculo y procesamiento. De hecho, los computadores fueron diseñados y contruídos para poder realizar operaciones matemáticas muchísimo más rápido que un ser humano. Sin embargo, es un ser humano el que le tiene que decir al computador, de alguna forma, qué operaciones debe realizar. A esto se le denomina “programar”. En este capítulo se introducirán los conceptos de algoritmo y programa, y se estudiará cómo programar el computador para que evalúe expresiones simples con distintos tipos de datos básicos.

1.1 ¿Qué es un algoritmo?

Un *algoritmo* es una secuencia finita de pasos que permiten ejecutar cualquier tarea (como por ejemplo, hacer un huevo frito). La palabra algoritmo viene de la transcripción latina del nombre de *Abu Abdallah Muhammad ibn Musa al-Khwarizmi*, un famoso matemático, astrónomo y geógrafo persa del siglo IX, padre del álgebra y quien introdujo el concepto matemático de algoritmo.

Podemos considerar que definir un algoritmo es la primera etapa en la resolución de un problema. Generalmente, procedemos como sigue:

- Identificar un problema
- Contextualizar los elementos que definen dicho problema
- Relacionar mediante pasos de ejecución los elementos para resolver el problema

Tal como vimos anteriormente, un algoritmo es la representación natural, paso a paso, de cómo podemos resolver un problema. Esto generalmente se conoce como una técnica de diseño *top-down*

(o de arriba hacia abajo). En otras palabras, partimos de un problema concreto y lo rompemos en unidades elementales que se pueden resolver paso a paso mediante alguna estrategia conocida de antemano.

Veamos a continuación un ejemplo: supongamos que queremos cocinar un huevo frito para acompañar un almuerzo. La definición del algoritmo que resuelve este problema sería:

- Problema: hacer un huevo frito
- Elementos: **huevo**, **aceite**, **cocina**, **fósforo**, **sartén**
- Pasos de ejecución:
 1. Encender un **fósforo**
 2. Con el **fósforo**, prender un quemador en la **cocina**
 3. Colocar la **sartén** sobre el quemador de la **cocina**
 4. Poner unas gotas de **aceite** sobre la **sartén**
 5. Tomar un **huevo** y quebrarlo
 6. Colocar el **huevo** quebrado sobre la **sartén**
 7. Esperar hasta que el **huevo** esté listo

1.2 ¿Qué es un programa?

Un *programa* es una especificación ejecutable de un algoritmo. Para representar un programa en un computador, utilizamos un *lenguaje de programación*. En el contexto de este curso, usaremos el lenguaje Python por su facilidad de aprendizaje.

Para comenzar a trabajar con Python utilizaremos su intérprete. El intérprete de Python es una aplicación que lee expresiones que se escriben, las evalúa y luego imprime en pantalla el resultado obtenido.

Es importante recalcar que utilizaremos en este curso a Python como una herramienta y no un fin. No hay que olvidar que el objetivo es aprender a resolver problemas, utilizando un computador como apoyo para realizar esta tarea.

1.3 Tipos de datos básicos

Un *tipo de datos* es un atributo que indica al computador (y/o al programador) algo sobre la clase de datos sobre los que se va a procesar. Esto incluye imponer restricciones en los datos, tales como qué valores pueden tomar y qué operaciones se pueden realizar. Todos los valores que aparecen en un programa tienen un tipo. A continuación, revisaremos algunos de los tipos de datos básicos con los que vamos a trabajar en este curso:

1.3.1 Enteros (int)

El tipo de datos *entero* representa un subconjunto finito de los números enteros. El número mayor que puede representar depende del tamaño del espacio usado por el dato y la posibilidad (o no) de representar números negativos. Las típicas operaciones aritméticas que se pueden realizar con estos datos son: suma, resta, multiplicación y división. Por ejemplo: $\dots, -3, -2, -1, 0, 1, 2, 3, \dots$

1.3.2 Reales (float)

El tipo de datos *real* permite representar una aproximación decimal de números reales. La aproximación depende de los recursos disponibles del computador, por lo que todas las operaciones entre valores reales son aproximaciones. Los números reales se escriben separando la parte entera de la parte decimal con un punto. Por ejemplo: 0.303456

1.3.3 Texto (str)

El tipo de datos *texto* (o *string*) permite representar cadenas de caracteres indicadas entre comillas simples (‘ ’) o dobles (“ ”). Por ejemplo: ‘hola’, ‘103’, ‘Mi mascota es un ñandú!’. Al respecto, hay que recalcar que la expresión ‘103’ es de tipo texto porque está entre comillas, aún cuando su contenido se puede entender como un número.

1.4 Programas simples

Como vimos, un programa es una representación ejecutable de un algoritmo. Esta representación está definida por una expresión que al ser evaluada genera un valor. A continuación veremos cómo definir y evaluar programas simples.

1.4.1 Evaluación de expresiones

Una *expresión* es una combinación entre valores y operadores que son evaluados durante la ejecución de un programa. Por ejemplo, la expresión $1 + 1$ es una expresión aritmética, tal que al evaluarla produce el valor 2.

Con los tipos de datos explicados anteriormente, podemos realizar operaciones entre ellos utilizando operadores específicos en cada caso. Así, para datos numéricos (enteros y reales), podemos usar los operadores de suma (+), resta (-), multiplicación (*) y división (/). La prioridad de estos operadores es la misma usada en álgebra: primero se evalúan los operadores multiplicativos de izquierda a derecha según orden de aparición (* y /), y luego los aditivos (+, -). En el caso de querer imponer una evaluación en particular que no siga el orden preestablecido, podemos indicarlo utilizando paréntesis. Por ejemplo:

```
1 >>> 3 + 5
2   -> 8
3 >>> 3 + 2 * 5
4   -> 13
5 >>> (3 + 2) * 5
6   -> 25
```

Nota: La secuencia de caracteres >>> significa que usamos el intérprete de Python. La línea siguiente, con la secuencia de caracteres -> indica la respuesta del intérprete, tal como Python la evalúa.

En Python, se definen dos operadores adicionales para operaciones matemáticas recurrentes: elevar a potencia (**) y calcular el resto de una división entera (%). La prioridad del operador % es la misma que la de los operadores multiplicativos, mientras que la del operador ** es mayor. Así:

```
1 >>> 2 ** 3
2   -> 8
3 >>> 4 ** 0.5
4   -> 2.0
```

```
5 >>> 10 % 3
6 -> 1
```

Para operar con valores de tipo texto, en Python utilizamos generalmente dos operadores: si queremos unir (concatenar) dos cadenas de texto, lo indicamos con el operador `+`; por otro lado, si queremos repetir una cadena de texto, lo indicamos con el operador `*`. Por ejemplo:

```
1 >>> 'abra' + 'cadabra'
2 -> 'abracadabra'
3 >>> 'ja' * 3
4 -> 'jajaja'
```

Finalmente, es importante notar que en Python los valores a los que se evalúa una expresión dependen del tipo de los operandos. Así, si la expresión está compuesta únicamente de números enteros, el resultado obtenido también será de tipo entero. Por ejemplo:

```
1 >>> 1 / 2
2 -> 0
```

En efecto, dado que 1 y 2 son valores de tipo entero, la división entre ellos también lo será (y, por ende, se transforma el valor obtenido al tipo entero). Si queremos calcular el valor real de dicha operación, debemos forzar a que al menos uno de los dos operandos sea real. Así:

```
1 >>> 1.0 / 2.0
2 -> 0.5
3 >>> 1 / 2.0
4 -> 0.5
5 >>> 1.0 / 2
6 -> 0.5
```

Finalmente, si queremos juntar valores de tipo texto con valores de tipo numérico, debemos convertir estos últimos previamente a valores de tipo texto. Por ejemplo, si queremos transformar un valor n a un valor equivalente de tipo texto, utilizamos la función de Python `str`. De igual manera, si tenemos un valor de tipo texto que se puede entender como número, por ejemplo `'103'`, podemos convertir el valor en tipo numérico usando las funciones `int` y `float`. Así:

```
1 >>> 'En el curso hay ' + str(100) + ' alumnos'
2 -> 'En el curso hay 100 alumnos'
3 >>> '100' + '1'
4 -> '1001'
5 >>> int('100') + 1
6 -> 101
```

1.4.2 Variables

Una *variable* es el nombre que se le da a un valor o a una expresión. El valor de una variable está dado por la definición más reciente del nombre. Para evaluar una expresión con variables, usamos una semántica por sustitución, esto es, reemplazamos en la expresión los valores asociados al nombre por aquellos que están definidos por la variable.

Por ejemplo, si la variable `x` tiene el valor 8, al evaluar la expresión `doble = 2 * x`, entonces la variable `doble` contendrá el valor 16 (pues `doble` $\rightarrow$ `2 * x` $\rightarrow$ `2 * 8` $\rightarrow$ 16).

Para crear variables en un programa podemos utilizar cualquier letra del alfabeto, o bien, una combinación de letras, números y el símbolo `_` siempre que el primer carácter no sea un número. Para *asignar* una variable a una expresión (o al resultado de ésta), utilizamos el operador `=`.

Notemos que es importante el orden en que se realiza la definición de variables y expresiones: la sintaxis correcta es `variable = expresión`, y no al revés. En Python se evalúa la expresión y se define la variable con el valor resultante. Por ejemplo:

```
1 >>> a = 8 # la variable a contiene el valor 8
2 >>> b = 12 # la variable b contiene el valor 12
3 >>> a # mostramos el valor de a
4 -> 8
5 >>> a + b # creamos una expresion y mostramos su valor
6 -> 20
7 >>> c = a + 2 * b # creamos una expresion, se evalua y se define c
8 >>> # con su valor
9 >>> c # mostramos el valor de c
10 -> 32
11 >>> a = 10 # redefinimos a
12 >>> a + b
13 -> 22
14 >>> c # el valor de c no cambia, pues lo calculamos antes de la
15 >>> # redefinicion de a
16 -> 32
```

En Python, el símbolo `#` sirve para introducir un comentario. Los comentarios son explicaciones que da el programador y que no son procesadas por el intérprete. Es importante utilizar comentarios para introducir aclaraciones relevantes en el código. Por otro lado, no es recomendable abusar de ellos! Es mejor utilizar nombres de variables que tengan un significado propio (relevante a la expresión que están calculando), y utilizar los comentarios sólo en situaciones especiales.

Veamos un mal ejemplo:

```
1 >>> a = 8
2 >>> b = 12
3 >>> c = a * b
```

En este ejemplo notamos que queremos calcular el área de un rectángulo, pero los nombres de las variables no son los indicados. En este caso, las variables `a`, `b` y `c` pueden representar cualquier cosa. Sería mucho más adecuado escribir:

```
1 >>> ancho = 8
2 >>> largo = 12
3 >>> area = ancho * largo
```

1.5 Errores

Hasta ahora hemos visto únicamente expresiones correctas, tal que al evaluarlas obtenemos un resultado. ¿Qué es lo que imprime el intérprete en este caso?

```
1 >>> dia = 13
2 >>> mes = 'agosto'
3 >>> 'Hoy es ' + dia + ' de ' + mes
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: cannot concatenate 'str' and 'int' objects
```

En este caso, el intérprete nos dice que hemos cometido un error en nuestro programa: “**cannot concatenate 'str' and 'int' objects**”, y es de tipo **TypeError**. Esto en español significa que estamos intentando unir valores de tipo texto con valores de tipo entero. Tal como lo vimos anteriormente, esto es incompatible para el intérprete por lo que debemos corregir la instrucción.

```
1 >>> dia = 13
2 >>> mes = 'agosto'
3 >>> 'Hoy es ' + str(dia) + ' de ' + mes
4 -> 'Hoy es 13 de agosto'
```

Existen distintos tipos de errores. Es útil conocerlos para no cometerlos, y eventualmente para saber cómo corregirlos.

1.5.1 Errores de sintaxis

Un *error de sintaxis* se produce cuando el código no sigue las especificaciones propias del lenguaje. Estos errores se detectan en el intérprete antes de que se ejecute, y se muestran con un mensaje de error indicando el lugar “aproximado” en el que se detectó la falta. Por ejemplo:

```
1 >>> numero = 15
2 >>> antecesor = (numero - 1))
```

```
File "<stdin>", line 1
    antecesor = (numero - 1))
                        ^
SyntaxError: invalid syntax
```

1.5.2 Errores de nombre

Un *error de nombre* se produce cuando utilizamos una variable que no se ha definido anteriormente en el programa. Por ejemplo:

```
1 >>> lado1 = 15
2 >>> area = lado1 * lado2
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'lado2' is not defined
```

1.5.3 Errores de tipo

Un *error de tipo* ocurre cuando aplicamos operaciones sobre tipos que no son compatibles. Por ejemplo:

```
1 >>> dia = 13
2 >>> mes = 'agosto'
3 >>> 'Hoy es ' + dia + ' de ' + mes
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: cannot concatenate 'str' and 'int' objects
```

1.5.4 Errores de valor

Un *error de valor* ocurre cuando, a pesar de tener una expresión bien formada, se aplican operaciones sobre valores que no corresponden al tipo en el que fueron definidas. Por ejemplo:

```
1 >>> nombre = 'Juan Soto'
2 >>> int(nombre)
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'Juan Soto'
```

1.5.5 Errores de indentación

Un *error de indentación* ocurre cuando los espacios en el código no son consistentes. Esto es porque Python usa los espacios para delimitar *bloques de código*, como veremos en el próximo capítulo.

```
1 >>> x = 3
2 >>>     x
```

```
File "<stdin>", line 1
  x
  ^
IndentationError: unexpected indent
```

1.5.6 Errores lógicos

En un programa no todos los errores pueden ser detectados por el computador. Los *errores lógicos* son aquellos que se producen por descuido del programador al escribir las instrucciones y pueden provocar resultados muy inesperados. Por ejemplo, estos errores se pueden producir al darle un nombre incorrecto o ambiguo a una variable o a otras situaciones más complejas.

Lamentablemente, el intérprete no nos indicará cuándo o en qué línea se producen estos errores, por lo que la mejor manera de enfrentarlos es evitarlos y seguir una metodología limpia y robusta que nos permita asegurar a cabalidad que lo que estamos escribiendo efectivamente es lo que esperamos que el computador ejecute. Por ejemplo:

```
1 >>> numero = 15
2 >>> doble = 3 * numero
3 >>> doble # esperaríamos 30, luego debe haber algun error en el código
4 -> 45
```

Capítulo 2

Funciones¹

La clase anterior vimos cómo crear expresiones sencillas que operan con números. El objetivo de esta clase es ir un paso más allá y desarrollar pequeños trozos de código que implementen operaciones como un conjunto. Al igual que en matemática, en computación llamamos *función* a una estructura que recibe valores de entrada y genera un valor de salida.

2.1 Variables y funciones

En los cursos de matemática aprendimos a relacionar cantidades mediante expresiones con variables. Por ejemplo, conocemos bien la relación entre el área de un círculo y su radio. Si el radio de un círculo está dado por la variable “r”, entonces su área se calcula mediante la expresión:

$$areaCirculo = 3.14 \cdot r^2$$

Así, si tenemos un círculo cuyo radio tiene valor igual a 5, sabemos que sustituyendo la variable “r” con este valor, obtenemos su área:

$$areaCirculo = 3.14 \cdot 5^2 = 3.14 \cdot 25 = 78.5$$

2.1.1 Definición de funciones

Al igual que en matemática, en computación una función es una regla que cumple con las mismas características, puesto que describe cómo producir información en base a otra que está dada como entrada. Luego, es importante que cada función sea nombrada de forma en que sea clara la relación entre su nombre y el objetivo que cumple. El ejemplo anterior puede transformarse en una función cuyo nombre sería `areaCirculo` y en Python estaría definida como sigue:

```
1 def areaCirculo(radio):  
2     return 3.14 * radio ** 2
```

La primera línea define la función, asignándole un nombre y los argumentos que recibe. Notemos que para declarar una función debemos utilizar la palabra clave `def` y terminar la declaración con el símbolo dos puntos (`:`). Los argumentos de una función son la información de entrada que ésta recibe, y que serán utilizados para evaluar las expresiones que la definen. En nuestro ejemplo, la función recibe un sólo argumento, que es el valor del radio del círculo que será utilizado para calcular su área.

¹Traducido al español y adaptado de: M. Felleisen et al.: *How to Design Programs*, MIT Press. Disponible en: www.htdp.org

La segunda línea define la salida de la función. La palabra clave `return` indica el fin de la función, y la expresión que la sucede será evaluada y retornada como la salida.

Para utilizar una función, debemos invocarla con el valor de sus argumentos. En nuestro ejemplo, para calcular el área de un círculo con radio igual a 5, invocamos a la función de la siguiente manera:

```
1 >>> areaCirculo(5)
2 -> 78.5
```

Internamente, lo que sucede es similar al reemplazo de variables que se explicó anteriormente. La última línea de la función es evaluada de la siguiente manera:

$$\text{areaCirculo}(5) \rightarrow 3.14 * 5 ** 2 \rightarrow 3.14 * 25 \rightarrow 78.5$$

Una función puede estar compuesta tanto de operadores básicos como también de otras funciones previamente definidas. Es decir, dada la definición de una función, ésta puede ser utilizada a su vez por otras funciones. Por ejemplo, imaginemos que queremos definir una función que calcule el área de un anillo. Dado que el área de un anillo se calcula restando área del círculo externo con el área del círculo interno, podemos utilizar nuestra función `areaCirculo` para implementar esta nueva función.



Para esto, debemos crear una función que reciba dos argumentos, el radio del círculo externo y el radio del círculo interno, calcule el área de ambos círculos y finalmente los reste. Luego, la función que calcula el área de un anillo queda definida de la siguiente manera:

```
1 def areaAnillo(exterior, interior):
2     return areaCirculo(exterior) - areaCirculo(interior)
```

Para utilizar nuestra función, podemos tomar como ejemplo un anillo con radio externo igual a 5 e interno igual a 3:

```
1 >>> areaAnillo(5, 3)
2 -> 50.24
```

En efecto, esta evaluación corresponde a:

$$\begin{aligned} \text{areaAnillo}(5, 3) &\rightarrow \text{areaCirculo}(5) - \text{areaCirculo}(3) \\ &\rightarrow 3.14 * 5 ** 2 - 3.14 * 3 ** 2 \\ &\rightarrow 3.14 * 25 - 3.14 * 9 \\ &\rightarrow 50.24 \end{aligned}$$

2.1.2 Indentación y subordinación de instrucciones

Es importante notar que en Python la indentación (cantidad de tabulaciones hacia la derecha) de cada instrucción determina el alcance al que pertenece. En el caso de nuestra función de ejemplo `areaCirculo`, podemos ver que la primera línea está al margen izquierdo mientras que la segunda está

separado del margen izquierdo por una tabulación. Esto indica que la segunda línea está subordinada a la función, lo que se traduce que esta instrucción es parte de la ella. Así, si una función está compuesta por muchas instrucciones, cada línea debe estar indentada al menos una tabulación hacia la derecha más que la línea que indica el nombre y argumentos de la función. Notemos entonces que la función de nuestro ejemplo puede ser reescrita de la siguiente manera:

```
1 def areaCirculo(radio):
2     pi = 3.14
3     return pi * radio * radio
```

Como se puede observar, las dos líneas que definen esta función tienen una indentación a la derecha, quedando ambas subordinadas a la función misma. En caso de no respetar esta regla, un error de indentación ocurre:

```
1 def areaCirculo(radio):
2     pi = 3.14
3     return pi * radio * radio
```

```
File "<stdin>", line 3
    return pi * radio * radio
    ^
IndentationError: unexpected indent
```

O, al olvidar de indentar:

```
1 def areaCirculo(radio):
2     pi = 3.14
3     return pi * radio * radio
```

```
File "<stdin>", line 2
    pi = 3.14
    ^
IndentationError: expected an indented block
```

2.1.3 Alcance de una variable

La definición de una variable dentro de una función tiene un alcance local. El significado de esta frase se explicará a continuación a través de ejemplos.

Imaginemos que declaramos una variable `a`. Si una función realiza alguna operación que requiere de esta variable, el valor utilizado será aquel que contiene la variable. Por ejemplo:

```
1 >>> a = 100
2 >>> def sumaValorA(x):
3 ...     return x + a
4 >>> sumaValorA(1)
5 -> 101
```

Sin embargo, una variable puede ser redefinida dentro de una función. En este caso, cada vez que se deba evaluar una expresión dentro de la función que necesite de esta variable, el valor a considerar será aquel definido dentro de la función misma. Además, la redefinición de una variable se hace de manera local, por lo que no afectará al valor de la variable definida fuera de la función. Esto se puede observar en el siguiente ejemplo:

```
1 >>> a = 100
2 >>> def sumaValorA(x):
3 ...     a = 200
4 ...     return x + a
5 >>> sumaValorA(1)
6 -> 201
7 >>> a
8 -> 100
```

Por otra parte, si el argumento de una función tiene el mismo nombre que una variable definida fuera de ésta, la función evaluará sus instrucciones con el valor del argumento, pues es la variable que está dentro de su alcance:

```
1 >>> a = 100
2 >>> def sumaValorA(a):
3 ...     return 1 + a
4 >>> sumaValorA(5)
5 -> 6
```

Finalmente, cualquier variable definida dentro de la función no existe fuera de ésta, ya que queda fuera de su alcance (recuerde que éste es local a la función). Por ejemplo:

```
1 >>> def sumaValorA(a):
2 ...     b = 100
3 ...     return a + b
4
5 >>> sumaValorA(50)
6 150
7 >>> b
```

```
Traceback (most recent call last):
  File "stdin", line 1, in <module>
    b
NameError: name 'b' is not defined
```

2.2 Problemas

Rara vez los problemas vienen formulados de tal manera que basta con traducir una fórmula matemática para desarrollar una función. En efecto, típicamente se tiene una descripción informal sobre una situación, la que puede incluir información ambigua o simplemente poco relevante. Así, la primera etapa de todo programador es extraer la información relevante de un problema y luego traducirlo en expresiones apropiadas para poder desarrollar un bloque de código. Consideremos el siguiente ejemplo:

“Genera S.A. le paga \$4.500 por hora a todos sus ingenieros de procesos recién egresados. Un empleado típicamente trabaja entre 20 y 65 horas por semana. La gerencia de informática le pide desarrollar un programa que calcule el sueldo de un empleado a partir del número de horas trabajadas.”

En la situación anterior, la última frase es la que indica cuál es el problema que queremos resolver: escribir un programa que determine un valor en función de otro. Más específicamente, el programa recibe como entrada un valor, la cantidad de horas trabajadas, y produce otro, el sueldo de un

empleado en pesos. La primera oración implica cómo se debe calcular el resultado, pero no lo especifica explícitamente. Ahora bien, en este ejemplo particular, esta operación no requiere mayor esfuerzo: si un empleado trabaja una cantidad h de horas, su sueldo será: $\$4.500 \cdot h$.

Ahora que tenemos una expresión para modelar el problema, simplemente creamos una función en Python para calcular los valores:

```
1 def sueldo(h):  
2     return 4500 * h
```

En este caso, la función se llama `sueldo`, recibe un parámetro `h` representando a la cantidad de horas trabajadas, y devuelve `4500 * h`, que corresponde al dinero que gana un empleado de la empresa al haber trabajado `h` horas.

2.3 Un poco más sobre errores

En el capítulo anterior se habló acerca de los distintos tipos de error que se pueden producir al generar programas de computación. En particular, se discutió sobre los errores de sintaxis, de nombre y lógicos. Ahora se discutirá sobre dos nuevos tipos de error: los errores en tiempo de ejecución y los errores de indentación.

2.3.1 Errores de ejecución

Existen maneras de cometer errores que el intérprete de Python no notará hasta que la expresión escrita sea evaluada. Un claro ejemplo es la división por cero. Para el intérprete de Python la expresión `1 / 0` representa la división entre dos números cualquiera, pero que al evaluarla se generará el error descrito a continuación:

```
1 >>> 1 / 0
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
ZeroDivisionError: integer division or modulo by zero
```

En Python este tipo de error se llama `ZeroDivisionError`, la que indica claramente la fuente de la falla.

Otra manera de obtener este tipo de error es invocando una función con un número equivocado de argumentos. Por ejemplo, si utilizamos la función `areaCirculo` con dos argumentos en vez de uno, recibiremos un mensaje de error que lo indica:

```
1 >>> areaCirculo(5,3)
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: areaCirculo() takes exactly 1 argument (2 given)
```

2.3.2 Errores de indentación

Como vimos anteriormente, la indentación de un conjunto de instrucciones en Python tiene una connotación semántica, no sólo sintáctica, puesto que indica la subordinación de una instrucción. Así, es posible generar errores cuando una instrucción no está indentada de manera correcta.

Imagine que se desea definir una función que calcule el área de un cuadrado dado el largo de uno de sus lados. Una implementación de esta función podría ser de la siguiente manera:

```
1 >>> def areaCuadrado(lado):  
2 ... return lado * lado
```

Sin embargo, al intentar evaluar la expresión anterior, se obtiene un mensaje de error que indica que se espera una indentación de la instrucción para poder definir la función de manera correcta:

```
File "<stdin>", line 2  
    return lado * lado  
    ^  
IndentationError: expected an indented block
```

Se nota que este error de indentación tiene un mensaje diferente que el error que vimos anteriormente (“IndentationError: unexpected indent”), dado que el error de indentación es distinto.

Capítulo 3

Receta de Diseño¹

En el capítulo anterior vimos que el desarrollo de una función requiere varios pasos. Necesitamos saber qué es lo relevante en el enunciado del problema y qué podemos ignorar. Además, necesitamos saber qué es lo que la función recibirá como parámetros, y cómo relaciona estos parámetros con la salida esperada. Además, debemos saber, o averiguar, si Python provee operaciones básicas para manejar la información que necesitamos trabajar en la función. Si no, deberíamos desarrollar funciones auxiliares que implementen dichas operaciones. Finalmente, una vez que tengamos desarrollada la función, necesitamos verificar si efectivamente realiza el cálculo esperado (para el cual efectivamente implementamos la función). Esto puede evidenciar errores de sintaxis, errores de ejecución, o incluso errores de diseño.

Para trabajar apropiadamente, lo mejor es seguir una *receta de diseño*, esto es, una descripción paso a paso de qué es lo que tenemos que hacer y en qué orden. Basándonos en lo que hemos visto hasta ahora, el desarrollo de un programa requiere al menos las siguientes cuatro actividades:

1. Entender el propósito de la función.
2. Dar ejemplos de uso de la función.
3. Probar la función.
4. Especificar el cuerpo de la función.

En las siguientes secciones estudiaremos en detalle cada una de estas cuatro actividades.

3.1 Entender el propósito de la función

El objetivo de diseñar una función es el crear un mecanismo que consume y produce información. Luego, deberíamos empezar cada función dándole un nombre significativo y especificando qué tipo de información consume y qué tipo de información produce. A esto lo llamamos *contrato*. Por ejemplo, supongamos que nos piden diseñar una función que calcule el área de un rectángulo. Supongamos que la función se llama `areaRectangulo`. Su contrato se define como:

```
1 # areaRectangulo: num num -> num
```

¹Traducido al español y adaptado de: M. Felleisen et al.: *How to Design Programs*, MIT Press. Disponible en: www.htdp.org

El contrato consiste en dos partes: la primera, a la izquierda de los dos puntos especifica el nombre de la función; la segunda, a la derecha de los dos puntos, especifica qué tipo de datos consume y qué es lo que produce. Los tipos de valores de entrada se separan de los de salida por una flecha. En el caso de nuestro ejemplo el tipo de datos que consume es de tipo numérico, es decir, puede ser de tipo entero (`int`) o real (`float`), por lo que lo representamos con la palabra `num`. El valor que se produce también es de tipo numérico, dado que es de tipo entero si es que ambos datos de entrada son enteros, o es de tipo real si es que al menos uno de los datos de entrada es un número real. En general, representaremos los tipos de datos que conocemos hasta el momento como sigue (se irán agregando otros a lo largo del curso):

- Tipo entero: se representa con la palabra `int`.
- Tipo real: se representa con la palabra `float`.
- Tipo numérico (real o entero): se representa con la palabra `num`.
- Tipo texto: se representa con la palabra `str`.

Por ejemplo, para la función `areaAnillo` del capítulo anterior, su contrato es:

```
1 # areaAnillo: num num -> float
```

dado que los datos de entrada son numéricos (enteros o reales), pero el dato de salida siempre será de tipo `float`.

Una vez que tenemos especificado el contrato, podemos agregar el *encabezado* de la función. Éste reformula el nombre de la función y le da a cada argumento un nombre distintivo. Estos nombres son variables y se denominan los *parámetros* de la función. Miremos con más detalle el contrato y el encabezado de la función `areaRectangulo`:

```
1 # areaRectangulo: num num -> num
2 def areaRectangulo(largo, ancho):
3     ...
```

Aquí especificamos que los parámetros (en orden) que recibe la función se llaman `largo` y `ancho`.

Finalmente, usando el contrato y los parámetros, debemos formular un *propósito* para la función, esto es, un comentario breve sobre qué es lo que la función calcula. Para la mayoría de nuestras funciones, basta con escribir una o dos líneas; en la medida que vayamos desarrollando funciones y programas cada vez más grandes, podremos llegar a necesitar agregar más información para explicar el propósito de una función. Así, hasta el momento llevamos lo siguiente en la especificación de nuestra función:

```
1 # areaRectangulo: num num -> num
2 # calcula el area de un rectangulo de medidas
3 # largo y ancho
4 def areaRectangulo(largo, ancho):
5     ...
```

3.2 Dar ejemplos de uso de la función

Para tener un mejor entendimiento sobre qué es lo que debe calcular la función, evaluamos ejemplos para valores de entrada significativos y determinamos manualmente cuál debe ser la salida esperada. Por ejemplo, la función `areaRectangulo` debe generar el valor 15 para las entradas 5 y 3. Así, la especificación de nuestra función queda de la forma:

```
1 # areaRectangulo:  num num -> num
2 # calcula el area de un rectangulo de medidas
3 # largo y ancho
4 # ejemplo: areaRectangulo(5, 3) debe producir 15
5 def areaRectangulo(largo, ancho):
6     ...
```

El hacer ejemplos ANTES DE ESCRIBIR EL CUERPO DE LA FUNCIÓN ayuda de muchas maneras. Primero, es la única manera segura de descubrir errores lógicos. Si usáramos la función una vez implementada para generar estos ejemplos, estaríamos tentados a confiar en la función porque es mucho más fácil evaluar la función que predecir qué es lo que efectivamente hace. Segundo, los ejemplos nos fuerzan a pensar a través del proceso computacional, el que, para casos más complejos que veremos más adelante, es crítico para el desarrollo del cuerpo de la función. Finalmente, los ejemplos ilustran la prosa informal del propósito de la función. Así, futuros lectores del código, tales como profesores, colegas, o incluso clientes, podrán entender cuál es el concepto que está detrás del código.

3.3 Probar la función

Antes de completar la definición de la función, debemos definir como probarla. El proceso de probar una función se llama *testeo* o *testing*, y cada prueba se conoce como *test*.

En cualquier función que desarrollemos, nos debemos asegurar que al menos calcula efectivamente el valor esperado para los ejemplos definidos en el encabezado. Para facilitar el testeo, podemos hacer uso del comando `assert` de Python para definir un caso de uso y compararlo con el valor esperado.

Así, por ejemplo, si queremos probar que un valor calculado de la función es igual a uno que calculamos manualmente, podemos proceder como sigue:

```
1 assert areaRectangulo(5, 3) == 15
```

En este caso, le indicamos a Python que evalúe la aplicación de la función `areaRectangulo` con los parámetros 5 y 3, y verifique si el resultado obtenido es efectivamente 15. Si ese es el caso, la función se dice que *pasa el test*. En caso contrario, Python lanza un error y es entonces indicio que debemos verificar con detalle nuestra función. Por ejemplo:

```
1 >>> assert areaRectangulo(5, 3) == 15
2 >>> assert areaRectangulo(5, 3) == 0
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AssertionError
```

Es importante recalcar que las pruebas que realizamos no pueden probar que una función produce salidas correctas para TODAS las entradas posibles. Esto se debe a que hay un número infinito de

combinaciones de valores de entrada para pasar como parámetros. Sin embargo, el testeo es una estrategia muy potente para verificar errores de sintaxis o de diseño en la función.

Para los casos en los que la función no pasa un test, debemos poner especial atención a los ejemplos que especificamos en el encabezado. En efecto, es posible que los ejemplos estén incorrectos, que la función tenga algún tipo de error, o incluso que tanto los ejemplos como la función tengan errores. En cualquier caso, deberíamos volver a revisar la definición de la función siguiendo los cuatro pasos anteriores.

3.4 Especificar el cuerpo de la función

Finalmente, debemos definir cuál es el cuerpo de la función. Esto es, debemos reemplazar los puntos suspensivos (...) de nuestra definición anterior por un conjunto de instrucciones que conformarán el cómo se procesarán los parámetros de entrada para producir la salida.

Notemos que podemos formular el cuerpo de la función únicamente si entendemos cómo la función calcula el valor de salida a partir del conjunto de valores de entrada. Así, si la relación entre las entradas y la salida están dadas por una fórmula matemática, basta con traducir esta expresión a Python. Si por el contrario nos enfrentamos a un problema verbal, debemos construir previamente la secuencia de pasos necesaria para formular la expresión.

En nuestro ejemplo, para resolver el problema basta con evaluar la expresión `largo * ancho` para obtener el área del rectángulo. Así, la traducción en Python de este proceso sería:

```
1 def areaRectangulo(largo, ancho):  
2     return largo * ancho
```

Finalmente, la definición completa de nuestra función siguiendo la receta de diseño es como sigue:

```
1 # areaRectangulo:  num num -> num  
2 # calcula el area de un rectangulo de medidas  
3 # largo y ancho  
4 # ejemplo: areaRectangulo(5, 3) debe producir 15  
5 def areaRectangulo(largo, ancho):  
6     return largo * ancho  
7 # Tests  
8 assert areaRectangulo(5, 3) == 15
```

Notemos que todas las funciones que se definan deben seguir la receta de diseño. Por ejemplo, si queremos definir la función `areaCuadrado` podemos seguir la receta de diseño y reutilizar la función `areaRectangulo`, obteniendo lo siguiente:

```
1 # areaCuadrado:  num -> num  
2 # calcula el area de un cuadrado de medida lado  
3 # ejemplo: areaCuadrado(5) debe producir 25  
4 def areaCuadrado(lado):  
5     return areaRectangulo(lado, lado)  
6 # Tests  
7 assert areaCuadrado(5) == 25
```


Capítulo 4

Módulos y Programas¹

En general, un programa consta no sólo de una, sino de muchas definiciones de funciones. Por ejemplo, si retomamos el ejemplo del anillo que vimos en el capítulo 2, tenemos dos funciones: una para calcular el área de un círculo (`areaCirculo`) y una para calcular el área del anillo propiamente tal (`areaAnillo`). En otras palabras, dado que la función `areaAnillo` retorna el valor que queremos en nuestro programa, decimos que es la *función principal*. De igual manera, dado que la función `areaCirculo` apoya a la función principal, decimos que es una *función auxiliar*.

El uso de funciones auxiliares hace que el diseño de programas sea más manejable, y deja finalmente al código más limpio y entendible de leer. Por ejemplo, consideremos las siguientes dos versiones para la función `areaAnillo`:

```
1 def areaAnillo(interior, exterior): # Buena practica
2     return areaCirculo(exterior) - areaCirculo(interior)
3
4 def areaAnillo(interior, exterior): # Mala practica
5     return 3.14 * exterior ** 2 - 3.14 * interior ** 2
```

La primera definición está basada en una *composición* de funciones auxiliares. El diseñarla de esta manera nos ayudó a descomponer el problema en subproblemas más pequeños, pero más abordables. De hecho, el sólo leer la definición de la función (sin siquiera saber cómo están definidas las funciones auxiliares) nos da a entender que para calcular el área del anillo basta con restar el área de un círculo externo con el área del círculo en su interior. Por el contrario, la definición de la segunda versión de nuestra función obliga al lector el reconstruir la idea de que las subexpresiones en efecto calculan el área de un círculo. Peor aún, ¡estamos escribiendo dos veces la misma expresión!

Para un programa pequeño como el que hemos visto en el ejemplo, las diferencias entre ambos estilos de diseño de funciones son menores, aun cuando bastante significativas. Sin embargo, para programas o funciones más grandes, el usar funciones auxiliares no se vuelve una opción, sino una necesidad. Esto es, cada vez que se nos pida escribir un programa, debemos considerar el descomponerlo en funciones, y éstas a su vez descomponerlas en funciones auxiliares hasta que cada una de ellas resuelva UNO Y SÓLO UN SUBPROBLEMA particular.

¹Parte de este texto fue traducido al español y adaptado de: M. Felleisen et al.: *How to Design Programs*, MIT Press. Disponible en: www.htdp.org

4.1 Descomponer un programa en funciones

Consideremos el siguiente problema:

“Una importante cadena de cines de Santiago tiene completa libertad en fijar los precios de las entradas. Claramente, mientras más cara sea la entrada, menos personas estarán dispuestas a pagar por ellas. En un reciente estudio de mercado, se determinó que hay una relación entre el precio al que se venden las entradas y la cantidad de espectadores promedio: a un precio de \$5.000 por entrada, 120 personas van a ver la película; al reducir \$500 en el precio de la entrada, los espectadores aumentan en 15. Desafortunadamente, mientras más personas ocupan la sala para ver la película, más se debe gastar en limpieza y mantenimiento general. Para reproducir una película, el cine gasta \$180.000. Asimismo, se gastan en promedio \$40 por espectador por conceptos de limpieza y mantenimiento. El gerente del cine le encarga determinar cuál es la relación exacta entre las ganancias y el precio de las entradas para poder decidir a qué precio se debe vender cada entrada para maximizar las ganancias totales.”

Si leemos el problema, está clara cuál es la tarea que nos piden. Sin embargo, no resulta del todo evidente el cómo hacerlo. Lo único de lo que podemos estar seguros es que varias cantidades dependen entre sí.

Cuando nos vemos enfrentados a estas situaciones, lo mejor es identificar las dependencias y ver las relaciones una por una:

- Las *ganancias* corresponden a la diferencia entre los ingresos y los gastos.
- Los *ingresos* se generan exclusivamente a través de la venta de entradas. Corresponde al producto del valor de la entrada por el número de espectadores.
- Los *gastos* están formados por dos ítems: un gasto fijo (\$180.000) y un gasto variable que depende del número de espectadores.
- Finalmente, el enunciado del problema también especifica cómo el número de espectadores depende del precio de las entradas.

Definamos, pues, una función por cada una de estas dependencias; después de todo, las funciones precisamente calculan cómo distintos valores dependen de otros. Siguiendo la receta de diseño que presentamos en el capítulo anterior, comenzaremos definiendo los contratos, encabezados y propósitos para cada una de las funciones:

```
1 # ganancias: int -> int
2 # calcular las ganancias como la diferencia entre los ingresos y
3 # los gastos dado precioEntrada
4 def ganancias(precioEntrada):
5     ...
```

Notemos que las ganancias totales dependen del precio de las entradas, dado que tanto los ingresos como los gastos dependen a su vez del precio de las entradas.

```
1 # ingresos: int -> int
2 # calcular el ingreso total, dado precioEntrada
3 def ingresos(precioEntrada):
4     ...
5
```

```
6 # gastos: int -> int
7 # calcular los gastos totales, dado precioEntrada
8 def gastos(precioEntrada):
9     ...
10
11 # espectadores: int -> int
12 # calcular el numero de espectadores, dado precioEntrada
13 def espectadores(precioEntrada):
14     ...
```

Esto nos permite enunciar la primera regla en el diseño de programas:

Antes de escribir cualquier línea de código siga la receta de diseño para cada función: formule el contrato, encabezado y propósito de la función, plantee ejemplos de uso relevantes y formule casos de prueba para verificar que su función se comportará correctamente.

Una vez escritas las formulaciones básicas de las funciones y al haber calculado a mano una serie de ejemplos de cálculo, podemos reemplazar los puntos suspensivos ... por expresiones de Python. En efecto, la función `ganancias` calcula su resultado como la diferencia entre los resultados arrojados por las funciones `ingresos` y `gastos`, tal como lo sugiere el enunciado del problema y el análisis de las dependencias que hicimos anteriormente. El cálculo de cada una de estas funciones depende del precio de las entradas (`precioEntrada`), que es lo que indicamos como parámetro de las funciones. Para calcular los ingresos, primero calculamos el número de espectadores para `precioEntrada` y lo multiplicamos por `precioEntrada`. De igual manera, para calcular los gastos sumamos el costo fijo al costo variable, que corresponde al producto entre el número de espectadores y 40. Finalmente, el cálculo del número de espectadores también se sigue del enunciado del problema: podemos suponer una relación lineal entre el número de espectadores y el valor de la entrada. Así, 120 espectadores están dispuestos a pagar \$5.000, mientras que cada \$500 que se rebajen del precio, vendrán 15 espectadores más.

La definición de las funciones es como sigue:

```
1 def ganancias(precioEntrada):
2     return ingresos(precioEntrada) - gastos(precioEntrada)
3
4 def ingresos(precioEntrada):
5     return espectadores(precioEntrada) * precioEntrada
6
7 def gastos(precioEntrada):
8     return 180000 + espectadores(precioEntrada) * 40
9
10 def espectadores(precioEntrada):
11     return 120 + (5000 - precioEntrada) * 15 / 500
```

Si bien es cierto que podríamos haber escrito directamente la expresión para calcular el número de espectadores en todas las funciones, esto es altamente desventajoso en el caso de querer modificar una parte en la definición de la función. De igual manera, el código resultante sería completamente ilegible. Así, formulamos la siguiente regla que debemos seguir junto con la receta de diseño:

APUNTE DE USO INTERNO

PROHIBIDA SU DISTRIBUCIÓN

Diseñe funciones auxiliares para cada dependencia entre cantidades mencionadas en la especificación de un problema y por cada dependencia descubierta al elaborar ejemplos de casos de uso. Siga la receta de diseño para cada una de ellas.

De igual manera, en ocasiones podemos encontrarnos con valores que se repiten varias veces en una misma función o programa. Claramente, si queremos modificar su valor, no nos gustaría tener que modificarlo en cada una de las líneas en que aparece. Luego, lo recomendable es que sigamos una *definición de variable*, en la que asociamos un identificador con un valor (de la misma manera que a una variable le asociamos el resultado de una expresión). Por ejemplo, podemos asociarle el valor 3.14 a una variable de nombre PI para referirnos al valor de π en todas las líneas que necesitemos en nuestro programa. Así:

PI = 3.14

Luego, cada vez que nos refiramos a PI, el intérprete reemplazará el valor por 3.14.

El usar nombres para las constantes hace más entendible el código para identificar dónde se reemplazan distintos valores. De igual manera, el programa se vuelve más mantenible en el caso de necesitar modificar el valor de la constante: sólo lo cambiamos en la línea en que hacemos la definición, y este cambio se propaga hacia abajo cada vez que se llama al identificador. En caso contrario, deberíamos modificar a mano cada una de las líneas en que escribimos directamente el valor.

Formulemos esta tercera regla:

Dé nombres relevantes a las constantes que utilizará frecuentemente en su programa, y utilice estos nombres en lugar de hacer referencia directa a su valor.

4.2 Módulos

La programación modular es una técnica de diseño que separa las funciones de un programa en *módulos*, los cuales definen una finalidad única y contienen todo lo necesario, código fuente y variables, para cumplirla. Conceptualmente, un módulo representa una separación de intereses, mejorando la mantenibilidad de un software ya que se fuerzan límites lógicos entre sus componentes. Así, dada una segmentación clara de las funcionalidades de un programa, es más fácil la búsqueda e identificación de errores.

Hasta el momento, solo hemos escrito programas en el intérprete de Python, por lo que no podemos reutilizar el código que hemos generado hasta el momento. Para guardar código en Python, lo debemos hacer en archivos con extensión `.py`. Así, basta con abrir un editor de texto (como por ejemplo el bloc de notas), copiar las funciones que deseamos almacenar y guardar el archivo con un nombre adecuado y extensión `.py`. Es importante destacar que existen muchas herramientas que destacan las palabras claves de Python con diferentes colores, haciendo más claro el proceso de escribir código.

Imaginemos ahora que queremos calcular el perímetro y el área de un triángulo dado el largo de sus lados. Primero debemos definir la función `perimetro` que recibe tres parámetros:

```
1 # perimetro: num num num -> num
2 # calcula el perimetro de un triangulo de lados a, b, y c
3 # ejemplo: perimetro(2, 3, 2) devuelve 7
```

```

4 def perimetro(a,b,c):
5     return a + b + c
6 # Test
7 assert perimetro(2, 3, 2) == 7

```

Dado que esta función pertenece a lo que se esperaría fueran las funcionalidades disponibles de un triángulo, crearemos un módulo que la almacene, cuyo nombre será `triangulo`. Así, abriremos un archivo con nombre `triangulo.py` y copiaremos nuestra función dentro de él.

Luego, solo nos queda definir la función de área. Sabemos que el área de un triángulo puede calcularse en función de su semiperímetro, representado por p , que no es más que la mitad del perímetro de un triángulo. La relación entre área y semiperímetro de un triángulo de lados a , b y c está dada por la siguiente fórmula:

$$A = \sqrt{p * (p - a) * (p - b) * (p - c)}$$

Para traducir esta fórmula en una función ejecutable necesitamos la función raíz cuadrada, que está incluida en el módulo `math` de Python. Para importar un módulo externo debemos incluir la siguiente línea en nuestro módulo triángulo: `import math`, que literalmente significa importar un módulo externo para ser usado en un programa. Para utilizar una función de un módulo, la notación a usar es `modulo.funcion(...)`. Luego, si la función raíz cuadrada del módulo `math` de Python se llama `sqrt` y toma un parámetro, podemos definir la función `area` de un triángulo de la siguiente manera:

```

1 # area: num num num -> float
2 # calcula el area de un triangulo de lados a,b, y c
3 # ejemplo: area(2, 3, 2) devuelve 1.98...
4 def area(a, b, c):
5     semi = perimetro(a, b, c) / 2.0
6     area = math.sqrt(semi * (semi - a) * (semi - b) * (semi - c))
7     return area
8
9 # Tests
10 assert area(3,4,5) == 6

```

Finalmente, nuestro módulo `triangulo` quedaría de la siguiente manera:

Contenido del archivo `triangulo.py`

```

1 import math
2
3 # perimetro: num num num -> num
4 # calcula el perimetro de un triangulo de lados a, b, y c
5 # ejemplo: perimetro(2, 3, 2) devuelve 7
6 def perimetro(a,b,c):
7     return a + b + c
8
9 # Test
10 assert perimetro(2, 3, 2) == 7
11
12
13 # area: num num num -> float
14 # calcula el area de un triangulo de lados a,b, y c

```

Contenido del archivo triangulo.py (cont)

```
15 # ejemplo: area(3,4,5) devuelve 6
16 def area(a, b, c):
17     semi = perimetro(a, b, c) / 2.0
18     area = math.sqrt(semi * (semi - a) * (semi - b) * (semi - c))
19     return area
20
21 # Test
22 assert area(3,4,5) == 6
```

4.3 Programas interactivos

Muchas veces se requiere crear programas que poseen algún tipo de interacción con el usuario. Por ejemplo, el usuario podría entregar el valor de los lados de un triángulo para calcular su perímetro o su área. En esta sección cubriremos dos conceptos básicos de cómo interactuar con un programa de software: pedir datos al usuario e imprimir mensajes en pantalla.

Para pedir datos al usuario, Python provee dos funciones: `input` y `raw_input`. La primera de ellas, `input`, recibe como parámetro un mensaje de tipo texto para el usuario y recupera el dato ingresado. Esto lo podemos ver en el siguiente ejemplo, en el cual se le pide al usuario ingresar un número:

```
1 >>> input('Ingrese un numero ')
2 -> Ingrese un numero 4
3 -> 4
```

Los valores ingresados por el usuario pueden ser guardados en variables. Con la función `input`, el tipo de la variable será el más adecuado a lo que ingrese el usuario. Es decir, si el usuario entrega un número, la variable será de tipo numérico, y si el usuario entrega una palabra o frase, la variable será de tipo texto. Veamos el ingreso de número:

```
1 >>> numero = input('Ingrese un numero ')
2 -> Ingrese un numero 10
3 >>> numero
4 -> 10
5 >>> doble = numero * 2
6 >>> doble
7 -> 20
```

El ingreso de texto es similar:

```
1 >>> nombre = input('Cual es su nombre?')
2 -> Cual es su nombre?'Enrique'
3 >>> nombre
4 -> 'Enrique'
```

Es importante notar que al ingresar valores de tipo texto, estos deben estar entre comillas para ser indentificados como tal por el intérprete. Cuando el usuario intenta ingresar texto sin comillas, el intérprete mostrará un error en pantalla.

La otra función para ingresar datos disponible en Python, `raw_input`, tiene un comportamiento similar, con la excepción de que todo valor ingresado se almacenará con tipo texto. Esto se ve en el siguiente código:

```
1 >>> numero = raw_input('Ingrese un numero')
2   -> Ingrese un numero10
3 >>> numero
4   -> '10'
5 >>> doble = numero * 2
6 >>> doble
7   -> '1010'
```

Para que la interacción entre el computador y el humano no sea solamente en una dirección, también es posible que el programa entregue información al usuario. Así, un programa puede desplegar información en la consola de Python usando la función `print`. Esta función se utiliza escribiendo su palabra clave, seguida del texto o número a imprimir, como se ve en el siguiente ejemplo:

```
1 >>> print 'Hola, mundo!'
2   -> Hola, mundo!
```

Cuando queremos mostrar más de un texto o número en una misma línea, por ejemplo dos frases seguidas, podemos unirlos por comas. Notemos que esto es equivalente a crear un elemento de tipo texto generado con el operador `+`. Para ver como funciona, preguntemos el nombre y el apellido al usuario, y luego mostrémoslo en pantalla.

```
1 >>> nombre = input('Cual es su nombre?')
2   -> Cual es su nombre?'Enrique'
3 >>> apellido = input('Cual es su apellido?')
4   -> Cual es su apellido?'Jorquera'
5 >>> print 'Su nombre es', nombre, apellido
6   -> Su nombre es Enrique Jorquera
```

Volvamos al ejemplo del inicio, en donde calculamos el perímetro y área de un triángulo. Ya que sabemos cómo preguntar información al usuario, sería interesante construir un programa que pregunte los lados de un triángulo al usuario y utilice nuestro módulo para calcular los valores de su perímetro y área.

Para realizar este programa, debemos realizar tres pasos:

1. Importar el módulo creado;
2. preguntar por los valores necesarios, en este caso los lados del triángulo;
3. utilizar el módulo `triangulo` para calcular el área y perímetro.

Así, primero que nada, debemos importar el módulo que creamos con la palabra clave `import`:

```
1 import triangulo
```

Luego, debemos preguntar por el largo de cada lado del triángulo y almacenarlos en variables cuyos nombres sean representativos, como se muestra a continuación:

```

1 print 'Calcular el area y perimetro de un triangulo'
2 l1 = input('Ingrese el largo del primer lado')
3 l2 = input('Ingrese el largo del segundo lado')
4 l3 = input('Ingrese el largo del tercer lado')

```

Y finalmente utilizamos nuestro módulo para calcular al área y perímetro del triángulo dado:

```

1 print 'El perimetro del triangulo es', triangulo.perimetro(l1, l2, l3)
2 print 'El area del triangulo es', triangulo.area(l1, l2, l3)

```

El programa resultante se puede ver a continuación:

```

1 import triangulo
2
3 print 'Calcular el area y perimetro de un triangulo'
4 l1 = input('Ingrese el largo del primer lado')
5 l2 = input('Ingrese el largo del segundo lado')
6 l3 = input('Ingrese el largo del tercer lado')
7
8 print 'El perimetro del triangulo es', triangulo.perimetro(l1, l2, l3)
9 print 'El area del triangulo es', triangulo.area(l1, l2, l3)

```

Ahora que tenemos listo nuestro programa, podemos guardarlo en un archivo .py y ejecutarlo cada vez que necesitemos calcular el área y perímetro de un triángulo cualquiera (suponiendo que los valores entregados corresponden a un triángulo válido).

Para terminar, una manera alternativa de importar una función de un módulo es ocupar la instrucción:

```

1 from nombreModulo import nombreFuncion

```

Note que con esta instrucción sólo se está importando la función `nombreFuncion` del módulo `nombreModulo`, y ninguna otra que pueda haber en dicho módulo. Además, para invocar esta función ya no se debe escribir `nombreModulo.nombreFuncion(...)`, sino que debe escribirse `nombreFuncion(...)`. Finalmente, si se desea importar de esta forma todas las funciones del módulo, se ocupa la instrucción:

```

1 from nombreModulo import *

```

Modificando nuestro programa interactivo para ocupar esta forma alternativa de importar funciones de un módulo, queda como sigue:

Contenido del archivo pruebaTriangulo.py

```

1 from triangulo import *
2
3 print 'Calcular el area y perimetro de un triangulo '

```


Contenido del archivo pruebaTriangulo.py (cont)

```
4 l1 = input ('Ingrese el largo del primer lado ')
5 l2 = input ('Ingrese el largo del segundo lado ')
6 l3 = input ('Ingrese el largo del tercer lado ')
7
8 print 'El perimetro del triangulo es ', perimetro(l1, l2, l3)
9 print 'El area del triangulo es ', area(l1, l2, l3)
```

Capítulo 5

Expresiones y Funciones Condicionales¹

En general, los programas deben trabajar con distintos datos en distintas situaciones. Por ejemplo, un videojuego puede tener que determinar cuál es la velocidad de un objeto en un rango determinado, o bien cuál es su posición en pantalla. Para un programa de control de maquinaria, una condición puede describir en qué casos una válvula se debe abrir. Para manejar condiciones en nuestros programas, necesitamos una manera de saber si esta condición será *verdadera* o *falsa*. Así, necesitamos una nueva clase de valores, los que, por convención, llamamos valores *booleanos* (o valores de verdad). En este capítulo veremos los valores de tipo booleano, expresiones que se evalúan a valores booleanos, y expresiones que calculan valores dependiendo del valor de verdad de una evaluación.

5.1 Valores booleanos

Consideremos el siguiente problema:

“Genera S.A. le paga \$4.500 por hora a todos sus ingenieros de procesos recién egresados. Un empleado típicamente trabaja entre 20 y 65 horas por semana. La gerencia de informática le pide desarrollar un programa que calcule el sueldo de un empleado a partir del número de horas trabajadas *si este valor está dentro del rango apropiado*.”

Las palabras en *cursiva* resaltan qué es lo nuevo respecto al problema que presentamos en el capítulo de Funciones. Esta nueva restricción implica que el programa debe manipular al valor de entrada de una manera si tiene una forma específica, y de otra manera si no. En otras palabras, de la misma manera que las personas toman decisiones a partir de ciertas condiciones, los programas deben ser capaces de operar de manera condicional.

Las condiciones no deberían ser nada nuevo para nosotros. En matemática, hablamos de proposiciones *verdaderas* y *falsas*, las que efectivamente describen condiciones. Por ejemplo, un número puede ser igual a, menor que, o mayor que otro número. Así, si x e y son números, podemos plantear las siguientes tres proposiciones acerca de x e y :

1. $x = y$: “ x es igual a y ”;
2. $x < y$: “ x es estrictamente menor que y ”;

¹Parte de este texto fue traducido al español y adaptado de: M. Felleisen et al.: *How to Design Programs*, MIT Press. Disponible en: www.htdp.org

3. $x > y$: “ x es estrictamente mayor que y ”.

Para cualquier par de números (reales), una y sólo una de estas tres proposiciones es verdadera. Por ejemplo, si $x = 4$ y $y = 5$, entonces la segunda proposición es verdadera y las otras son falsas. Si $x = 5$ y $y = 4$, entonces la tercera es verdadera y las otras son falsas. En general, una proposición es verdadera para ciertos valores de variables y falsa para otros.

Además de determinar si una proposición atómica es verdadera o falsa en algún caso, a veces resulta importante determinar si la combinación de distintas proposiciones resulta verdadera o falsa. Consideremos las tres proposiciones anteriores, las que podemos combinar, por ejemplo, de distintas maneras:

1. $x = y$ y $x < y$ y $x > y$;
2. $x = y$ o $x < y$ o $x > y$;
3. $x = y$ o $x < y$.

La primera proposición compuesta es siempre falsa, pues dado cualquier par de números (reales) para x e y , dos de las tres proposiciones atómicas son falsas. La segunda proposición compuesta es, sin embargo, siempre verdadera para cualquier par de números (reales) x e y . Finalmente, la tercera proposición compuesta es verdadera para ciertos valores y falsa para otros. Por ejemplo, es verdadera para $x = 4$ y $y = 4$, y para $x = 4$ y $y = 5$, mientras que es falsa si $x = 5$ y $y = 3$.

Al igual que en matemática, Python provee comandos específicos para representar el valor de verdad de proposiciones atómicas, para representar estas proposiciones, para combinarlas y para evaluarlas. Así, el valor lógico *verdadero* es **True**, y el valor *falso* se representa por **False**. Si una proposición relaciona dos números, esto lo podemos representar usando *operadores relacionales*, tales como: `==` (igualdad), `>` (mayor que), y `<` (menor que).

Traduciendo en Python las tres proposiciones matemáticas que definimos inicialmente, tendríamos lo siguiente:

1. `x == y`: “ x es igual a y ”;
2. `x < y`: “ x es estrictamente menor que y ”;
3. `x > y`: “ x es estrictamente mayor que y ”.

Además de los operadores anteriores, Python provee como operadores relacionales: `<=` (menor o igual que), `>=` (mayor o igual que), y `!=` (distinto de).

Una expresión de Python que compara números tiene un resultado, al igual que cualquier otra expresión de Python. El resultado, sin embargo, es **True** o **False**, y no un número. Así, cuando una proposición atómica entre dos números es verdadera, en Python se evalúa a **True**. Por ejemplo:

```
1 >>> 4 < 5
2 -> True
```

De igual manera, una proposición falsa se evalúa a **False**:

```
1 >>> 4 == 5
2 -> False
```

Para expresar condiciones compuestas en Python usaremos tres conectores lógicos: `and` (conjunción lógica: “y”), `or` (disyunción lógica: “o”) y `not` (negación: “no”). Por ejemplo, supongamos que queremos combinar las proposiciones atómicas `x == y` y `y < z`, de tal manera que la proposición compuesta sea verdadera cuando ambas condiciones sean verdaderas. En Python escribiríamos:

```
1 x == y and y < z
```

para expresar esta relación. De igual manera, si queremos formular una proposición compuesta que sea verdadera cuando (al menos) una de las proposiciones sea verdadera, escribimos:

```
1 x == y or y < z
```

Finalmente, si escribimos algo como:

```
1 not x == y
```

lo que estamos indicando es que deseamos que la negación de la proposición sea verdadera.

Las condiciones compuestas, al igual que las condiciones atómicas, se evalúan a `True` o `False`. Consideremos por ejemplo la siguiente condición compuesta: `5 == 5 and 5 < 6`. Note que está formada por dos proposiciones atómicas: `5 == 5` y `5 < 6`. Ambas se evalúan a `True`, y luego, la evaluación de la compuesta se evalúa a: `True and True`, que da como resultado `True` de acuerdo a las reglas de la lógica proposicional. Las reglas de evaluación para `or` y `not` siguen el mismo patrón.

En las siguientes secciones veremos por qué es necesario formular condiciones para programar y explicaremos cómo hacerlo.

5.2 Funciones sobre booleanos

Consideremos la siguiente función sencilla para verificar una condición sobre un número:

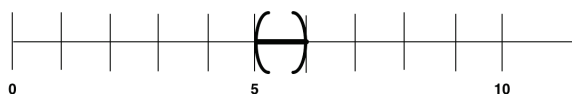
```
1 # esIgualA5: num -> bool
2 # determinar si n es igual a 5
3 def esIgualA5(n):
4     return n == 5
```

Esta función produce `True` si y sólo si su argumento es igual a 5. Su contrato contiene un nuevo elemento: la palabra `bool`. Al igual que `int`, `float` y `str`, la palabra `bool` representa una clase de valores *booleanos* que está definida en Python. Sin embargo, a diferencia de los valores de tipo numérico y texto, los booleanos sólo pueden ser `True` o `False`.

Consideremos este ejemplo un poco más sofisticado:

```
1 # estaEntre5y6: num -> bool
2 # determinar si n esta entre 5 y 6 (sin incluirlos)
3 def estaEntre5y6(n):
4     return 5 < n and n < 6
```

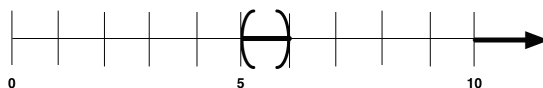
Es claro ver que esta función consume un número (entero o real) y produce `True` si el número está entre 5 y 6, sin incluirlos. Una forma de entender el funcionamiento de esta función es describiendo el intervalo que definen las condiciones en la recta numérica, tal como lo muestra la siguiente figura.



De igual manera, si queremos operar con condiciones más complejas sobre números, un primer paso puede ser determinar los rangos de definición en la recta numérica, y luego definir una función sobre los valores que se pueden tomar en el (los) intervalo(s). Por ejemplo, la siguiente función determina si un número está entre 5 o 6, o bien es mayor que 10:

```
1 # estaEntre5y6MayorQue10: num -> bool
2 # determinar si n esta entre 5 y 6 (sin incluirlos),
3 # o bien es mayor o igual que 10
4 def estaEntre5y6MayorQue10(n):
5     return estaEntre5y6(n) or n >= 10
```

Y la figura que representa las porciones de la recta numérica donde la función se evalúa a **True** es:



Esto es, cualquier número entre 5 y 6 sin incluirlos, o bien, cualquier número mayor o igual que 10. En este caso, desarrollamos una condición compuesta *componiendo* los distintos trozos que definen al intervalo donde la función se debe evaluar a *verdadero*.

5.3 Condiciones

Imaginemos que queremos crear un programa que juegue al cachipún con el usuario, pero que siempre le gane, independiente de lo que éste le entregue. Esto significa que debemos diseñar e implementar un programa que, dada una jugada del usuario, entregue la jugada que le gana según las reglas del cachipún. Para esto, en las dos secciones siguientes veremos las *expresiones condicionales* y las instrucciones que provee Python para crear funciones con este tipo de expresiones.

Las *expresiones condicionales* se caracterizan por ser del tipo:

si pregunta **entonces** respuesta

En particular para Python, la traducción de estas expresiones está dada de la siguiente manera:

```
1 if pregunta:
2     respuesta
```

Al ser ejecutada, se verifica si el resultado de la evaluación de la pregunta es verdadero o falso. En Python, esto quiere decir si el valor evaluado es igual a **True** o **False**. Por ejemplo, imaginemos que tenemos dos variables de tipo numérico y queremos saber si son iguales. Si las variables se llaman **x** e **y**, podemos mostrar en pantalla al usuario si es que esta condición es verdadera:

```
1 if x == y:
2     print 'Son iguales!'
```

Una expresión condicional puede estar compuesta de más de una pregunta asociada a una respuesta. En el ejemplo anterior, podríamos además decir cuál de las dos variables representa al número mayor. Así, las expresiones condicionales también pueden ser de la forma:

```

    si pregunta entonces respuesta,
    sino pregunta entonces respuesta
    ...
    sino pregunta entonces respuesta

```

En Python, para modelar este tipo de expresiones podemos utilizar las instrucciones `elif` y `else`, como se muestra a continuación:

```

1 if pregunta:
2     respuesta
3 elif pregunta:
4     respuesta
5 ...
6 elif pregunta:
7     respuesta

```

O:

```

1 if pregunta:
2     respuesta
3 elif pregunta:
4     respuesta
5 ...
6 else:
7     respuesta

```

Al igual que en las expresiones condicionales, los tres puntos indican que las expresiones `if` pueden tener más de una condición. Las expresiones condicionales, como ya hemos visto, se componen de dos expresiones **pregunta** y una **respuesta**. La pregunta es una expresión condicional que al ser evaluada siempre debe entregar un valor booleano, y la respuesta es una expresión que sólo será evaluada si es que la condición asociada a esta se cumple.

Tomemos el ejemplo anterior, en donde comparamos las variables de tipo numérico `x` e `y`, y mostremos al usuario cuál de los dos es mayor o, en su defecto, si son iguales. Así, tenemos tres casos posibles: (i) ambas variables representan a números del mismo valor; (ii) `x` tiene un valor mayor a `y`; (iii) `y` tiene un valor mayor a `x`. La traducción de estas tres condiciones en el lenguaje Python está dado como sigue:

```

1 if x == y:
2     print 'Son iguales!'
3 elif x > y:
4     print 'x es mayor que y'
5 elif y > x:
6     print 'y es mayor que x'

```

O:

```

1 if x == y:
2     print 'Son iguales!'

```

```

3 elif x > y:
4     print 'x es mayor que y'
5 else:
6     print 'y es mayor que x'

```

Cuando se evalúa una expresión condicional completa, esto se hace en orden, evaluando cada pregunta, o condición, una por una. Si una pregunta se evalúa como verdadero, entonces la respuesta asociada a esa pregunta se evaluará y será el resultado de la expresión condicional completa. Si no es así, entonces se continuará con la evaluación de la siguiente pregunta y así sucesivamente hasta que alguna de las condiciones se cumpla. Esto quiere decir que para el ejemplo anterior, primero se evaluará la primera pregunta (`x == y`) y si esta se cumple, se mostrará en consola el mensaje `'Son iguales!'`. Si es que no se cumple, entonces seguirá con la siguiente instrucción `elif` y evaluará su pregunta asociada, `x > y`, imprimiendo en pantalla si es que esta condición se cumple. Si no, evaluará la última pregunta e imprimirá el mensaje.

Aunque las expresiones del ejemplo anterior tienen una sintaxis algo diferente, ambas son equivalentes. Podemos notar que la expresión de la derecha está formada solamente con instrucciones `if` y `elif`, lo que significa que se evalúan las tres condiciones posibles de nuestro ejemplo de manera explícita. Mientras que la expresión de la derecha utiliza la instrucción `else`, la cual indica que su respuesta será evaluada sólo si ninguna de las preguntas anteriores se evalúa como verdadera.

Volvamos a nuestro ejemplo del cachipún en el cual el usuario siempre pierde. Para diseñar un programa que determine la jugada ganadora dada una entrada del usuario, debemos identificar las tres situaciones posibles, resumidas a continuación:

- Si el usuario entrega piedra, el programa debe entregar papel
- Si el usuario entrega papel, el programa debe entregar tijera
- Si el usuario entrega tijera, el programa debe entregar piedra

Luego, el programa completo consta de tres partes principales: (i) pedir al usuario la jugada a ingresar; (ii) identificar la jugada que le ganará a la ingresada por el jugador humano; y por último (iii) mostrarla en pantalla. La segunda parte estará definida en una función que, dada una entrada, entregue como resultado la jugada ganadora. Así, siguiendo la receta de diseño, debemos, primero que todo, escribir su contrato y formular su propósito:

```

1 # jaliscoCachipun: str -> str
2 # entrega la jugada ganadora del cachipun dada una entrada valida
3 def jaliscoCachipun(jugada):
4     ...

```

Luego, debemos agregar un ejemplo de la función:

```

1 # jaliscoCachipun: str -> str
2 # entrega la jugada ganadora del cachipun dada una entrada valida
3 # ejemplo: jaliscoCachipun('tijera') debe producir 'piedra'
4 def jaliscoCachipun(jugada):
5     ...

```

A continuación, escribimos un test para probar que nuestra función se comporta de manera adecuada.

```

13 assert jaliscoCachipun('tijera') == 'piedra'

```

Finalmente, debemos especificar el cuerpo de la función:

```
4 def jaliscoCachipun(jugada):
5     if jugada == 'piedra':
6         return 'papel'
7     elif jugada == 'papel':
8         return 'tijera'
9     elif jugada == 'tijera':
10        return 'piedra'
```

La definición completa de nuestra función esta dada como sigue:

Contenido del archivo cachipun.py

```
1  # jaliscoCachipun: str -> str
2  # entrega la jugada ganadora del cachipun dada una entrada valida
3  # ejemplo: jaliscoCachipun('tijera') debe producir 'piedra'
4  def jaliscoCachipun(jugada):
5      if jugada == 'piedra':
6          return 'papel'
7      elif jugada == 'papel':
8          return 'tijera'
9      elif jugada == 'tijera':
10         return 'piedra'
11
12  #test
13  assert jaliscoCachipun('tijera') == 'piedra'
```

Ahora que nuestra función está completa, podemos usarla para jugar con el usuario:

Contenido del archivo juegoCachipun.py

```
1  from cachipun import jaliscoCachipun
2
3  print 'Juego del Jalisco cachipun'
4  jugada = input('Ingrese una jugada (piedra, papel o tijera)')
5  jugadaGanadora = jaliscoCachipun(jugada)
6  print ('Yo te gano con ' + jugadaGanadora)
```

5.4 Bloques de código condicionales en Python

Dado que la respuesta de una expresión condicional puede estar compuesta por más de una línea de código, es necesario indicar a qué pregunta pertenecen. Para esto, todas las líneas de código que pertenezcan a la respuesta de una expresión condicional deben estar indentadas un nivel más que la instrucción condicional a la que pertenecen. En otras palabras, todas las líneas que tengan una

indentación más que la cláusula `if` están subordinadas a esta, y se dice que forman un *bloque de código*. Un bloque de código sólo será evaluado si es que condición asociada se cumple.

En el ejemplo que vimos anteriormente, si no agregamos la indentación correspondiente se producirá un error al intentar ejecutar el programa:

```
1 >>> if x == y:
2 ... print 'son iguales!'
```

```
File "<stdin>", line 2
    print 'son iguales!'
    ^
IndentationError: expected an indented block
```

Esto es similar al comportamiento de Python cuando uno define funciones.

5.5 Diseñar funciones condicionales

Tal como vimos anteriormente, la clave para diseñar funciones que requieran expresiones condicionales es reconocer que el enunciado del problema genera casos e identificar cuáles son. Para enfatizar la importancia de esta idea, introduciremos y discutiremos la receta de diseño para las funciones condicionales. La nueva receta introduce un nuevo paso, llamado *análisis de los datos*, la cual requiere que un programador entienda las diferentes situaciones que se discuten en el enunciado del problema. También modifica los pasos de Ejemplo y Cuerpo de la receta de diseño explicada en los capítulos anteriores.

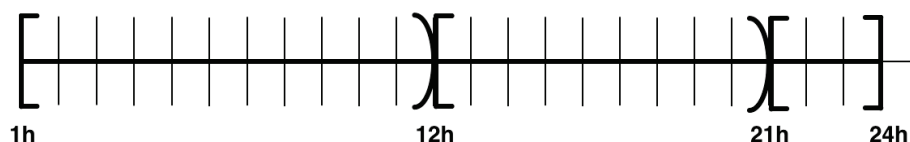
5.5.1 Análisis de los datos y definición

Luego de determinar que en el enunciado de un problema debemos lidiar con diferentes situaciones, es necesario identificar cada una de ellas.

Para funciones numéricas, una buena estrategia es dibujar una recta numérica e identificar los intervalos correspondientes a la situación particular a estudiar. Imaginemos que queremos implementar un programa que retorne el saludo correspondiente a la hora del día. Así, si son más de las 1 de la mañana y menos de las 12 de la tarde, el programa responderá 'Buenos días!'; si menos de las 21 horas, el mensaje será 'Buenas tardes!'; y si es más de las 21, entonces el programa deseeará las buenas noches. Consideremos el contrato de esta función:

```
1 # saludo: int -> int
2 # Determinar el saludo adecuado a la hora del día 1 <= h <= 24
3 def saludo(hora):
4     ...
```

Esta función recibe como entrada números enteros que están dentro del rango descrito por el contrato, y genera respuestas para tres diferentes situaciones, indicados por los intervalos de la siguiente figura:



5.5.2 Dar ejemplos de uso la función

Los ejemplos a escoger para este paso en la receta de diseño deben considerar las diferentes situaciones posibles. Como mínimo, debemos desarrollar un ejemplo por cada situación. Si describimos cada situación como un intervalo numérico, los ejemplos también deben incluir todos los casos de borde.

Para nuestra función `saludo`, deberíamos usar 1, 12, y 21 como casos de borde. Además, deberíamos escoger números como 8, 16, y 22 para probar el comportamiento al interior de cada uno de los tres intervalos.

5.5.3 El cuerpo de la función: diseñar condiciones

El cuerpo de la función debe estar compuesta de una instrucción `if` que tiene tantas cláusulas como situaciones diferentes. Este requerimiento sugiere de inmediato el siguiente cuerpo para nuestra función:

```
1 if (...):
2     ...
3 elif (...):
4     ...
5 elif (...):
6     ...
```

Luego formulamos las condiciones para describir cada una de las situaciones. Las condiciones son proposiciones sobre los parámetros de la función, expresados con operadores relacionales o con funciones hechas por nosotros mismos.

Las líneas de nuestro ejemplo se completa para traducirse en las siguientes tres condiciones:

1. $(1 \leq \text{hora})$ y $(\text{hora} \leq 12)$
2. $(12 < \text{hora})$ y $(\text{hora} \leq 21)$
3. $(21 < \text{hora})$

Agregando estas condiciones a la función, tenemos una mejor aproximación de la definición final:

```
1 def saludo(hora):
2     if (1 <= hora) and (hora <= 12):
3         ...
4     elif (12 < hora) and (hora <= 21):
5         ...
6     elif (21 < hora):
7         ...
```

En este punto, el programador debe asegurarse que las condiciones escogidas distinguen las diferentes entradas posibles de manera correcta. En particular, que cada dato posible esté dentro de una posible situación o intervalo. Esto quiere decir que cuando una pregunta o condición son evaluadas como `True`, todas las condiciones precedentes deben ser evaluadas como `False`.

5.5.4 El cuerpo de la función: responder a cada condición

Finalmente, debemos determinar qué debe producir la función por cada una de las cláusulas `if`. Más concretamente, debemos considerar cada expresión `if` por separado, asumiendo que la condición se

cumple.

En nuestro ejemplo, los resultados son especificados directamente del enunciado del problema. Estos son 'Buenos días!', 'Buenas tardes!', y 'Buenas noches!'. En ejemplos más complejos, debe ser el programador quien determina la expresión la respuesta de cada condición, puesto que no siempre están descritas de manera tan explícita. Estas se pueden contruir siguiendo los pasos de la receta de diseño que hemos aprendido hasta ahora.

```
1 def saludo(hora):
2     if (1 <= hora) and (hora <= 12):
3         return 'Buenos días!'
4     elif (12 < hora) and (hora <= 21):
5         return 'Buenas tardes!'
6     elif (21 < hora):
7         return 'Buenas noches!'
```

5.5.5 Simplificar condiciones

Cuando la definición de una función está completa y probada, un programador querrá verificar si es que las condiciones pueden ser simplificadas. En nuestro ejemplo, sabemos que la `hora` es siempre mayor o igual a uno, por lo que la primera condición podría ser formulada como:

```
hora <= 12
```

Más aún, sabemos que las expresiones `if` son evaluadas secuencialmente. Esto es, cuando la segunda condición es evaluada, la primera ya debe haber producido `False`. Por lo tanto sabemos que en la segunda condicional la cantidad *no es* menor o igual a 12, lo que implica que su componente izquierda es innecesaria. La definición completa y simplificada de la función `saludo` se describe como sigue:

Funcion saludo completa, y simplificada

```
1 # saludo: int -> str
2 # Determinar el saludo adecuado a la hora del dia 1 <= h <= 24
3 # ejemplos:
4 #     saludo(11) debe devolver 'Buenos días!'
5 #     saludo(15) debe devolver 'Buenas tardes!'
6 def saludo_simple(hora):
7     if (hora <= 12):
8         return 'Buenos días!'
9     elif (hora <= 21):
10        return 'Buenas tardes!'
11    elif (21 < hora):
12        return 'Buenas noches!'
13
14 # test:
15 assert saludo_simple(11) == 'Buenos días!'
16 assert saludo_simple(15) == 'Buenas tardes!'
```

¿Sería correcto el ejemplo? En realidad, no lo es. Si revisan los intervalos en la figura, se van a dar cuenta que el programa devuelve un saludo equivocado para la 12, entre otros problemas. ¡Esto

muestra la importancia de agregar tests ANTES de escribir el cuerpo de la función!

La versión correcta (y con condiciones simplificadas) de la funcion de saludo es la siguiente:

Funcion saludo testeada, depurada, y simplificada

```
1  # saludo: int -> str
2  # Determinar el saludo adecuado a la hora del dia 1 <= h <= 24
3  # ejemplos:
4  #     saludo(11) debe devolver 'Buenos dias!'
5  #     saludo(15) debe devolver 'Buenas tardes!'
6  def saludo_depurado(hora):
7      if (hora < 12):
8          return 'Buenos dias!'
9      elif (hora < 21):
10         return 'Buenas tardes!'
11     else:
12         return 'Buenas noches!'
13
14  # test:
15  assert saludo_depurado(1) == 'Buenos dias!'
16  assert saludo_depurado(11) == 'Buenos dias!'
17  assert saludo_depurado(12) == 'Buenas tardes!'
18  assert saludo_depurado(15) == 'Buenas tardes!'
19  assert saludo_depurado(21) == 'Buenas noches!'
20  assert saludo_depurado(23) == 'Buenas noches!'
21  assert saludo_depurado(24) == 'Buenas noches!'
```

Capítulo 6

Recursión

Muchas veces nos tocará enfrentarnos con definiciones que dependen de sí mismas. En particular, en programación se habla de funciones y estructuras recursivas cuando su definición depende de la misma definición de éstas. En este capítulo veremos un par de ejemplos de funciones recursivas.

6.1 Potencias, factoriales y sucesiones

Para calcular la potencia de un número con exponente entero, a^b , podemos usar la siguiente definición:

$$a^b = \begin{cases} 1 & \text{si } b = 0 \\ a \cdot a^{b-1} & \text{si } b > 0 \end{cases}$$

Como vemos, cuando el exponente es mayor que 0, para calcular la potencia necesitamos la misma definición con un exponente menor. La evaluación una potencia termina en el caso en que el exponente es 0. Por ejemplo, si queremos calcular 2^4 , basta con aplicar la definición:

$$\begin{aligned} 2^4 &= 2 \cdot 2^{4-1} \\ &= 2 \cdot 2^3 \\ &= 2 \cdot (2 \cdot 2^2) \\ &= 2 \cdot (2 \cdot (2 \cdot 2^1)) \\ &= 2 \cdot (2 \cdot (2 \cdot (2 \cdot 2^0))) \\ &= 2 \cdot (2 \cdot (2 \cdot (2 \cdot 1))) \\ &= 16 \end{aligned}$$

Una función que se define en términos de sí misma es llamada *función recursiva*.

Observe que si quitamos la primera parte de la definición de potencia, al calcular 2^4 , su evaluación nunca llegará a término. Esta parte es necesaria para dar término a la evaluación de una función recursiva, a la cual llamaremos *caso base*.

La segunda parte de la definición, a la cual llamaremos *caso recursivo*, es la que hace uso de su propia definición para continuar la evaluación hasta llegar al caso base.

Veamos cómo queda la función `potencia` escrita en Python:

```

1 # potencia: num int -> num
2 # calcula la potencia de base elevado a exp
3 # ejemplo: potencia(2, 4) devuelve 16
4 def potencia(base, exp):
5     if exp == 0:
6         # caso base
7         return 1
8     else:
9         # caso recursivo
10        return base * potencia(base, exp - 1)
11
12 # Test
13 assert potencia(2, 4) == 16
14 assert potencia(-1, 5) == -1
15 assert potencia(3, 0) == 1

```

Para entender cómo definir una función recursiva será clave la etapa de entender el propósito de la función. Como vimos, una función cumple el objetivo de consumir y producir información. Si en el proceso de consumir información se llega al mismo problema inicial (usualmente con una entrada o un parámetro más pequeño), entonces una solución recursiva puede ser correcta. En el ejemplo de la potencia, vimos que, por definición, debemos multiplicar la base por otro valor, que resulta ser la misma potencia con un exponente más pequeño, en cuyo caso conviene utilizar una solución recursiva.

De manera similar podemos definir el factorial de un número entero n :

$$n! = \begin{cases} 1 & \text{si } n = 0 \\ n \cdot (n-1)! & \text{si } n > 0 \end{cases}$$

```

1 # factorial: int -> int
2 # calcula el factorial de n
3 # ejemplo: factorial(10) devuelve 3628800
4 def factorial(n):
5     if n == 0:
6         # caso base
7         return 1
8     else:
9         # caso recursivo
10        return n * factorial(n - 1)
11
12 # Test
13 assert factorial(0) == 1
14 assert factorial(5) == 120
15 assert factorial(10) == 3628800

```

Otro ejemplo clásico de recursión es la generación de los números de Fibonacci. Los números de Fibonacci forman una sucesión de números que parten de la siguiente forma:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Se puede apreciar que cada número de la sucesión es igual a la suma de los dos números anteriores. Por supuesto, los dos primeros números de Fibonacci (0 y 1) son parte de la definición (caso base) de la sucesión, dado que no hay dos números anteriores para formarlos. El n -ésimo número de Fibonacci se calcula sumando los dos números anteriores de la sucesión, por lo que la recursión es clara en este caso. Formalmente, cada número de Fibonacci se puede calcular siguiendo esta definición:

$$F_n = \begin{cases} n & \text{si } 0 \leq n \leq 1 \\ F_{n-1} + F_{n-2} & \text{si } n > 1 \end{cases}$$

La implementación en Python de una función que calcula el n -ésimo número de Fibonacci es la siguiente:

```

1  # fibonacci: int -> int
2  # calcula el n-esimo numero de la sucesion de fibonacci
3  # ejemplo: fibonacci(7) devuelve 13
4  def fibonacci(n):
5      if n < 2:
6          # caso base
7          return n
8      else:
9          # caso recursivo
10         return fibonacci(n - 1) + fibonacci(n - 2)
11
12 # Test
13 assert fibonacci(0) == 0
14 assert fibonacci(1) == 1
15 assert fibonacci(7) == 13

```

6.2 Torres de Hanoi

Un ejemplo de recursión más complicado es el problema de las Torres de Hanoi.

Las Torres de Hanoi es el nombre de un puzzle matemático que consiste en mover todos los discos de una vara a otra, bajo ciertas restricciones. El juego consta de una plataforma con tres varas y n discos puestos en orden decreciente de tamaño en una de ellas. El objetivo del juego es mover todos los discos de una vara a la otra, de forma que al final se mantenga el mismo orden.

Las reglas del juego son las siguientes:

1. Sólo 1 disco puede ser movido a la vez.
2. No puede haber un disco más grande encima de uno más pequeño.
3. Un movimiento consiste en mover un disco en la cima de una pila de discos hacia otra pila de discos puestos en otra vara.

Nos interesa saber cuántos movimientos son necesarios para resolver el juego.

Solución

La clave para resolver el puzzle no está en determinar cuáles son los movimientos a realizar, sino en que el juego puede ser descompuesto en *instancias* más pequeñas. En el caso de las Torres de Hanoi, el problema está en mover n discos. Por lo tanto, veamos una forma de resolver el problema de forma de tener que resolver el juego con $n - 1$ discos, y volvamos a aplicar el procedimiento hasta mover todos los discos.

El objetivo del juego es mover la pila completa de una vara a la otra. Por lo que, inicialmente, lo único a lo que podemos apuntar a lograr es a trasladar el disco más grande de su vara a otra, y no nos

queda otra opción que mover todos los discos restantes de su vara a otra.

Supongamos que ya tenemos una función `hanoi(n)` que nos dice cuántos movimientos hay que realizar para mover n discos de una vara a otra. Esa función es la que queremos definir, ¡pero al mismo tiempo la necesitamos para resolver el problema! En el ejemplo de la figura necesitamos 15 movimientos para resolver el puzzle.

En resumen, debemos considerar los siguientes movimientos:

- Para mover el disco más grande de una vara a otra, necesitamos mover los $n - 1$ discos anteriores a otra vara, lo cual nos toma `hanoi(n-1)` movimientos.
- Luego, debemos mover el disco más grande de su vara a la desocupada, esto nos toma 1 movimiento.
- A continuación, debemos volver a mover los $n - 1$ discos restantes para que queden encima del disco grande que acabamos de mover. Esto nuevamente nos toma `hanoi(n-1)` movimientos.
- En total, necesitamos $2 \times \text{hanoi}(n-1) + 1$ movimientos para n discos.

¿Cuál es el caso base? Si tenemos 1 disco, sólo debemos moverlo de su vara a la otra para completar el juego.

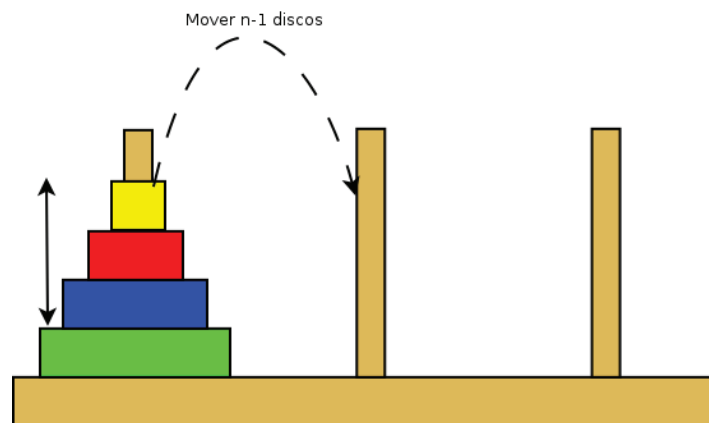


Figura 6.1: Mover los $n - 1$ primeros discos, recursivamente hacia la segunda vara.

Ahora que entendimos el propósito y la solución del juego, podemos escribirla en Python:

```

1  # hanoi:  int -> int
2  # calcula la cantidad de movimientos necesarios para resolver
3  # las Torres de Hanoi con n discos y 3 varas
4  # ejemplo: hanoi(4) devuelve 15
5  def hanoi(n):
6      if n == 1:
7          # caso base
8          return 1
9      else:
10         # caso recursivo
11         return 2 * hanoi(n - 1) + 1

```

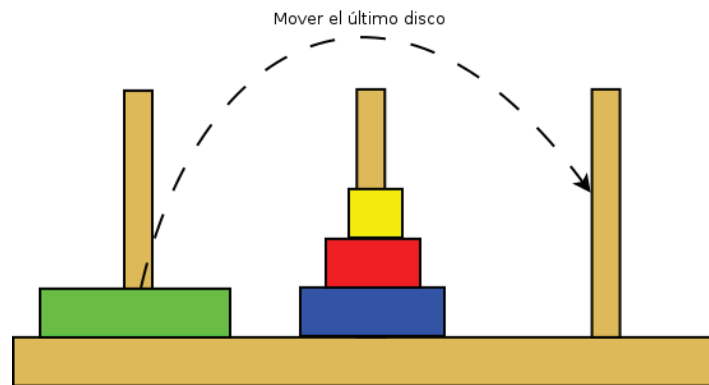
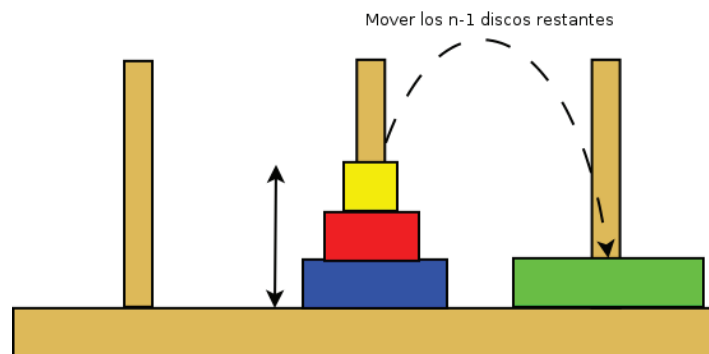



Figura 6.2: Mover el último disco hacia la tercera vara.

Figura 6.3: Volver a mover los primeros $n - 1$ discos, recursivamente hacia la tercera vara.

```

12
13 # Test
14 assert hanoi(1) == 1
15 assert hanoi(4) == 15
16 assert hanoi(5) == 31

```

Se puede apreciar que la solución de Hanoi sigue un patrón especial. De hecho, la solución a la ecuación de recurrencia $h(n) = 2 \cdot h(n - 1) + 1$ es $h(n) = 2^n - 1$, por lo que si hubiéramos llegado a ese resultado, podríamos utilizar la función **potencia** para resolver el problema.

6.3 Copo de nieve de Koch

Otro ejemplo interesante de recursión es el copo de nieve de Koch. El copo de nieve de Koch es un fractal cuya forma es similar a un copo de nieve. En la Figura 6.4 se puede apreciar un ejemplo.

El objetivo es describir el contorno de la figura hasta cierto nivel (puesto que el perímetro de la figura final es infinito). Para esto, es necesario describir un poco más en detalle la figura.

El fractal se genera a partir de un triángulo equilátero de lado s . A $s/3$ de distancia de un vértice se genera otro triángulo equilátero, de lado $s/3$. A distancia $(s/3)/3$ del vértice del último triángulo, se vuelve a generar otro más, y así sucesivamente. En la Figura 6.5 se pueden apreciar cuatro iteraciones del proceso.

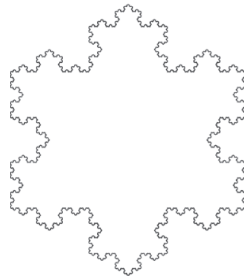


Figura 6.4: Copo de nieve de Koch.

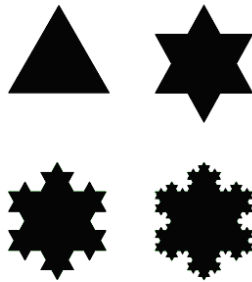


Figura 6.5: 4 primeras iteraciones de la generación del fractal. Fuente: Wikipedia.

¿Cómo podemos describir su contorno de manera recursiva? No es difícil observar que al generar la figura, al estar parados en algún triángulo de alguna iteración, a $1/3$ del lado de distancia de un vértice comenzamos a generar ¡la misma figura! El caso base lo debemos definir nosotros, puesto que sin él, la figura se irá generando indefinidamente.

Para programar nuestro copo de nieve en Python, utilizaremos el módulo **Turtle** que viene incluido en el lenguaje. El módulo **Turtle** provee varias funciones que dibujan en la pantalla. Conceptualmente, se trata de una tortuga robótica que se mueve en la pantalla marcando una línea a su paso. Algunas de las funciones provistas son:

- **turtle.forward(size)**: Se mueve **size** píxeles en su dirección.
- **turtle.left(angle)**, **turtle.right(angle)**: La tortuga gira a la izquierda o a la derecha, respectivamente, dependiendo de su sentido, en **angle** grados.
- **turtle.speed(speed)**: Se establece la velocidad de la tortuga. El parámetro **speed = 0** indica que se mueve a la máxima velocidad.
- **turtle.done()**: Se le indica que se han terminado las instrucciones para la tortuga.

Con estas funciones podemos indicarle cómo dibujar un fractal. Sin importar dónde comencemos, debemos dibujar un triángulo equilátero y al avanzar $1/3$ de su lado, se dibuja un fractal nuevamente. La Figura 6.6 muestra cómo debería quedar nuestra implementación.

Analicemos el proceso de dibujar el copo de nieve. Observe que el copo de nieve se trata de dibujar un triángulo equilátero, por lo que podemos dividir el problema en dibujar sólo un lado (puesto que los otros dos son iguales, salvo por el ángulo de donde viene). Supongamos que tenemos una función **snowflake** que dibuja los tres lados. Cada lado debe ser dibujado usando la curva de Koch.

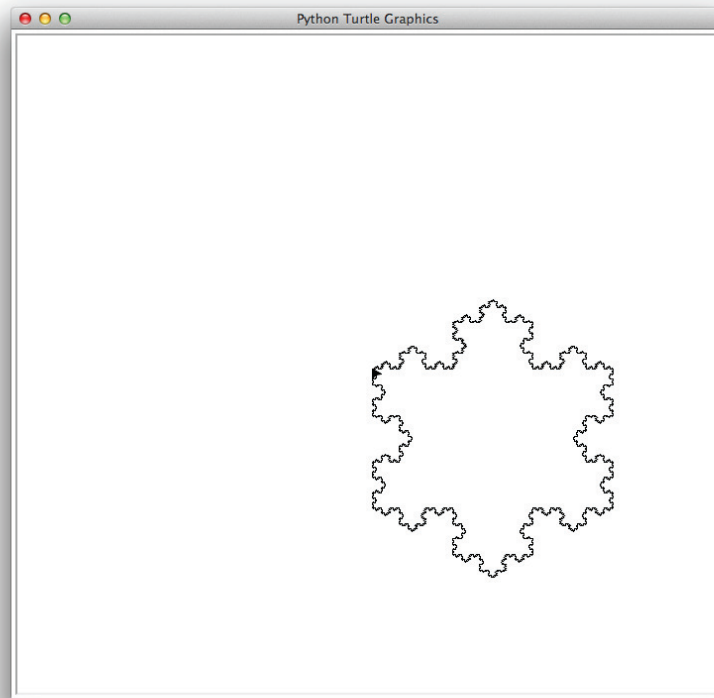


Figura 6.6: El resultado de dibujar el copo de nieve con Turtle

La función que dibuja cada lado la llamaremos `koch`, que representará la curva de Koch. Esta función debe dibujar cada lado del triángulo actual. Esta función debería recibir el lado del triángulo inicial y el caso base, es decir, el tamaño mínimo a partir del cual no debe continuar la recursión. Llamemos a estos parámetros `size` y `min_size`, respectivamente.

Una implementación posible es la siguiente:

1. Avanzar $1/3 \times \text{size}$ en la dirección actual.
2. Girar a la izquierda 60 grados.
3. Dibujar la misma curva de Koch de lado $1/3 \times \text{size}$.
4. Girar a la derecha 120 grados.
5. Dibujar la misma curva de Koch de lado $1/3 \times \text{size}$.
6. Girar a la izquierda 60 grados.
7. Avanzar $1/3 \times \text{size}$ en la dirección actual.

Sin embargo, tiene un error. Si dibujamos esta curva tres veces y creamos el triángulo con `snowflake`, resultará en lo que se puede apreciar en la Figura 6.7. Al separar nuestra función en dos, una que dibuja un lado y la otra que usa la primera para dibujar los tres lados, hemos perdido información. En particular, los vértices de los triángulos que se forman indirectamente al juntar las tres curvas (que son iguales al primero, al ser equiláteros del mismo tamaño) no generan sub-triángulos y no se forma la figura del copo de nieve. Para esto debemos generar más sub-triángulos incluso en

esos vértices.

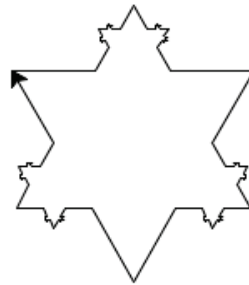


Figura 6.7: Primer intento del copo de nieve, juntando las tres curvas de Koch de la primera implementación.

Para solucionar este problema, modifiquemos nuestro algoritmo para que sólo dibuje una línea recta cuando llegamos al límite del tamaño:

1. Si `min_size ≥ size / 3`, avanzar `size` en la dirección actual.
2. Si no:
 - Dibujar la misma curva de Koch de lado $1/3 \times \text{size}$.
 - Girar a la izquierda 60 grados.
 - Dibujar la misma curva de Koch de lado $1/3 \times \text{size}$.
 - Girar a la derecha 120 grados.
 - Dibujar la misma curva de Koch de lado $1/3 \times \text{size}$.
 - Girar a la izquierda 60 grados.
 - Dibujar la misma curva de Koch de lado $1/3 \times \text{size}$.

Al graficar esta implementación, resultará en lo que se puede ver en la Figura 6.8.



Figura 6.8: La curva de Koch para un lado del triángulo.

La implementación en Python final:

Contenido del archivo koch.py

```
1 import turtle
2
3 # koch: int int -> None
```

Contenido del archivo koch.py (cont)

```

4  # dibuja la curva de koch de largo size
5  # y largo minimo min_size
6  # ejemplo: koch(320, 1)
7  def koch(size, min_size):
8      if (size / 3 < min_size):
9          # caso base
10         turtle.forward(size)
11     else:
12         # caso recursivo
13         koch(size / 3, min_size)
14         turtle.left(60)
15         koch(size / 3, min_size)
16         turtle.right(120)
17         koch(size / 3, min_size)
18         turtle.left(60)
19         koch(size / 3, min_size)
20
21 # snowflake: int int ->
22 # dibuja el copo de nieve de Koch
23 # de un triangulo de lado size
24 # y lado minimo min_size
25 # ejemplo: snowflake(320, 1)
26 def snowflake(size, min_size):
27     koch(size, min_size)
28     turtle.right(120)
29     koch(size, min_size)
30     turtle.right(120)
31     koch(size, min_size)
32
33 # ejemplo de uso
34 turtle.speed(0)
35 snowflake(320, 1)
36 turtle.done()

```

6.4 Receta de diseño para la recursión

Cuando escribimos funciones recursivas, es necesario seguir una versión distinta de la receta de diseño. En efecto, antes de escribir el código, tenemos que seguir varios pasos bien definidos.

1. Escribir varios ejemplos de uso de la función (incluyendo parámetros y resultado).
2 potencia 4 = 16
2 potencia 0 = 1
2 potencia 3 = 8
2. Decidir cual de los argumentos va a tener una descomposición recursiva.
El argumento con el procedimiento recursivo es la potencia
3. Entender cual es el caso de base para este argumento.
Cuando la potencia es igual a 0, el resultado es uno

4. Entender cual es el paso recursivo, como se descompone el problema
Uno resta una a la potencia, y multiplica
5. Nombrar cada pieza del problema
Tenemos una “base”, y una “potencia”
6. Aplicar el problema a uno de los ejemplos
 - (a) Tomar un ejemplo: **2 potencia 3 = 8**
 - (b) Determinar los parametros y el resultado
Los parametros son 2 y 3, el resultado 8
 - (c) Determinar cuales de los parametros son piezas del problema
2 es la base, y 3 es la potencia
 - (d) Cual es la repuesta para la llamada recursiva?
Para reducir el problema, tenemos que restar uno a la potencia, entonces la llamada recursiva es: 2 potencia 2 = 4
 - (e) Determinar como se combinan los elementos para determinar la respuesta final
dado que 2 potencia 2 = 4, multiplicamos este resultado por la base (2), y tenemos el resultado final, 8
7. Ocupar la receta de diseño normal, tomando en cuenta que el patrón de base es una función condicional, una rama siendo el caso de base, la otra siendo el caso recursivo.

Capítulo 7

Testing y Depuración de Programas

En este capítulo veremos formas y técnicas de testing y depuración de programas, basándonos en el diseño por contrato visto en el capítulo de Receta de Diseño.

Tal como vimos en aquel capítulo, es necesario probar que la función que definamos cumpla con el contrato estipulado. Estas pruebas deben asegurar con suficiente certeza de que la función cumple su objetivo de acuerdo a los parámetros ingresados. En este punto el contrato es muy importante, ya que especifica los tipos de datos y sus dominios que serán considerados dentro de la función. No es factible probar todos los posibles parámetros de una función, pero el contrato disminuye considerablemente estas opciones. De los casos restantes, debemos escoger sólo los casos más representativos.

Por ejemplo, consideremos la función `maximo`, que tiene el siguiente contrato:

```
1 # maximo: num num -> num
2 # devuelve el mayor de ambos numeros, a y b
3 # ejemplo: maximo(2, 4) devuelve 4
4 def maximo(a, b):
5     ...
```

El contrato establece que los parámetros de la función deben ser numéricos, por lo que el siguiente no sería un buen test:

```
1 assert maximo('hola', 5) == 'hola'
```

En cambio, este sería un buen test para la función:

```
1 assert maximo(10, 20) == 20
```

Consideremos la función `potencia` definida en el capítulo de Recursión. Esta función acepta como parámetros un número como base, y un entero como exponente. Podríamos probar distintos casos de esta forma:

```
1 assert potencia(2, 4) == 16
2 assert potencia(1.5, 3) == 3.375
3 assert potencia(10, 3) == 1000
```

Sin embargo, tal como definimos `potencia`, tanto en código como matemáticamente, hay casos llamados *de borde*, es decir, extremos dentro del dominio de datos que acepta, que serían relevantes

de usar en los tests. En el caso de la potencia, la definición cambia de acuerdo al valor del exponente, por lo que el caso de borde sería un buen test para nuestra función:

```
1 assert potencia(10000000, 0) == 1
```

Si tenemos una función, sea o no recursiva, con casos de borde o extremos dentro del dominio de datos, debemos testearla en esos casos. En general, debemos probar nuestra función en **casos representativos**. Por ejemplo, en `potencia`, el primer y tercer test son redundantes. El segundo es relevante ya que usamos otro tipo de datos para la base. El último test definido también es relevante, ya que prueba un caso de borde.

7.1 Afirmaciones (*assertions*)

Hasta el momento hemos visto usar la sentencia `assert` (afirmación) de Python para probar nuestras funciones. La sintaxis específica es la siguiente:

```
1 assert <condicion>
```

Como observación, `assert` no es una expresión. No puede ser evaluada y asignada a una variable, sino que puede ser vista como una sentencia o una palabra clave de Python que realiza una acción. Por otra parte, `<condicion>` sí corresponde a una expresión, por lo que se puede usar cualquier expresión que se evalúe a un valor de tipo `Boolean` en el lugar de `<condicion>`. Por ejemplo:

```
1 assert True
2 assert 10 < 12
3 assert a == b and (c < d or a < b)
```

Cuando la condición se evalúa a `True`, la afirmación no hace nada y el programa puede continuar. Cuando la condición evalúa a `False` (es decir, si no se cumple), la afirmación arroja un error y el programa termina:

```
1 >>> assert False
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    AssertionError
```

Esto es útil para probar el buen funcionamiento de nuestras funciones. Claramente, el comportamiento de las afirmaciones sugiere que Ud. debe escribirlas **antes** de escribir el código de su función, con valores que Ud. haya calculado antes. Recuerde que el código es una representación de su solución, no la solución.

El uso de distintos operadores lógicos para `assert` nos ayuda a escribir mejores tests. Observe que el uso de más operadores no es una característica especial de la afirmación, sino que corresponde a que la condición que se pasa es una expresión, y en ella se puede usar cualquier operador booleano.

En particular, podemos utilizar operadores no sólo de igualdad, sino también de comparación (`<` o `>`, etc). Por ejemplo, suponga que tenemos una función que calcula aleatoriamente un número entre 1 y 10. ¿Cómo hacemos un test para eso, si cada evaluación de la función resultará en un valor distinto? En este caso, la observación clave está en que no nos importa qué valor tome la función, sino el *rango* de valores. Por ejemplo:


```

1 import random
2 # escogeAlAzar: -> int
3 # Devuelve un numero al azar entre 1 y 10
4 def escogeAlAzar():
5     # random.random() retorna un numero al azar entre 0 y 1
6     return int(10 * random.random()) + 1
7
8 # Test
9 assert escogeAlAzar() <= 10
10 assert escogeAlAzar() >= 1

```

Si nuestra función retorna de un tipo booleano, no es necesario indicar la igualdad. Si tenemos una función `esPar` que retorna `True` si su argumento es un entero par, y `False` si no, puede ser probada de la siguiente forma:

```

1 assert esPar(4)
2 assert esPar(5) == False

```

Dado que `esPar` evalúa a un booleano, no es necesario indicar la igualdad. Si retorna `False`, es necesario indicarla, puesto que en caso contrario la afirmación arrojará error al no cumplirse.

7.2 Testear con números reales

En Python, y en la gran mayoría de los lenguajes de programación, los números reales son representados en el computador utilizando *aritmética de punto flotante*. Los números reales son infinitos, pero una máquina es finita y por lo tanto es necesario representar sólo un subconjunto finito de los números reales en un computador.

Estos números están sujetos a *errores de precisión*. Es decir, algunas operaciones no serán exactas debido a que no existe una representación para *cada* número posible. Puede intentar este ejemplo en el intérprete:

```

1 >>> 0.1 + 0.2
2 -> 0.30000000000000004

```

Los errores de precisión también pueden propagarse. Por ejemplo, considere dos cantidades, a_0 y a_1 , que en su representación en un computador poseen errores e_0 y e_1 respectivamente (como $0.1+0.2$ en Python). Si multiplicamos estos valores, el error se amplifica:

$$(a_0 + e_0) \cdot (a_1 + e_1) \approx a_0 a_1 + a_0 e_1 + a_1 e_0$$

Con esto en mente, ¿cómo podemos estar seguros de que una función que manipule números de punto flotante está correcta con respecto a nuestras pruebas? Para esto utilizamos una tolerancia que denotaremos ε , que nos servirá para comparar dentro de un rango de valores.

Implementemos la función *distancia euclidiana* que calcula la distancia entre dos puntos dados por sus coordenadas, x_0, y_0, x_1, y_1 .

```

1 # distancia: num num num num -> num
2 # calcula la distancia euclidiana entre (x0, y0) y (x1, y1)
3 # ejemplo: distancia(1, 0, 4, 0) devuelve 3.0
4 def distancia(x0, y0, x1, y1):

```

```

5     dx = (x1 - x0)
6     dy = (y1 - y0)
7     return (dx ** 2 + dy ** 2) ** 0.5

```

Consideremos las siguientes expresiones:

```

1 >>> d1 = distancia(0.1, 0.2, 0.2, 0.1)
2   -> 0.14142135623730953
3 >>> d2 = distancia(1, 2, 2, 1)
4   -> 1.4142135623730951
5 >>> 10 * d1 == d2
6   -> False

```

Conceptualmente, la última expresión debía ser verdadera, pero fue evaluada a `False` por errores de precisión. Para esto usamos el valor de tolerancia ϵ para evitar estos problemas:

```

1 # cerca: num num num -> bool
2 # retorna True si x es igual a y con
3 # precision epsilon
4 def cerca(x, y, epsilon):
5     diff = x - y
6     return abs(diff) < epsilon

```

El valor de `epsilon` dependerá de cuánta precisión necesitemos para nuestro programa. La función `abs` devuelve el valor absoluto de su argumento: `abs(1) == 1`, `abs(-1) == 1`, etc. Se necesita esta función porque sino un valor muy grande de `d2` podría hacer verdadera la afirmación, aun cuando lógicamente sea falsa. Ahora podemos definir nuestros tests:

```

1 # Tests
2 tolerancia = 0.0001
3 assert cerca(distancia(0, 0, 4, 0), 4.0, tolerancia)
4 assert cerca(distancia(0, 1, 1, 0), 1.4142, tolerancia)

```

7.3 Ejemplo: cálculo de la raíz cuadrada

Suponga que requiere evaluar dentro de una expresión la raíz cuadrada de un número. El módulo `math` de Python provee la función `math.sqrt`, pero para efectos de este ejercicio supondremos que no disponemos de `math`.

Herón de Alejandría, un matemático griego del siglo I, propuso un método para calcular una aproximación de la raíz cuadrada de un número. El algoritmo está basado en la siguiente idea general: si z es una estimación de $\sqrt{x}$, pero superior a $\sqrt{x}$, entonces el valor de x/z sería una estimación menor que $\sqrt{x}$. Si uno calcula el promedio de estas dos estimaciones, se logra una estimación más cercana del valor de $\sqrt{x}$. Por otra parte, si z es inferior a $\sqrt{x}$, entonces x/z sería superior a $\sqrt{x}$, y el promedio entre x y z estaría más cerca del valor de $\sqrt{x}$. El método de Herón consiste en repetir este proceso hasta alcanzar una precisión suficiente, lo que en el siglo I, sin computadores, era un proceso muy largo de realizar a mano. Por suerte, ¡nosotros tenemos hoy en día computadores para ayudarnos con los cálculos!

Vamos a diseñar una función recursiva para calcular la raíz cuadrada de un número:

- Los argumentos de la función recursiva son: el número positivo x , una estimación de $\sqrt{x}$, y un nivel de precisión `epsilon`. Para verificar que x es un número positivo utilizaremos una *precondición*, que implica agregar un test dentro de la función. En Python esto se puede implementar con `assert`.
- Caso base: si el cuadrado de la estimación está a distancia `epsilon` de x , se retorna el valor de la estimación. Para esto, utilizaremos la función `cerca`.
- Caso recursivo: Se calcula una mejor estimación de acuerdo al método de Heron, y se realiza el llamado recursivo.
- La función recursiva se llamará `heron_r` y será una función auxiliar a la función `heron`, que hará el primer llamado a la función recursiva con una estimación inicial de $\sqrt{x}$. Nota: el elegir como estimación un valor cercano a $\sqrt{x}$ hace que el programa termine más rápido, pero usar un valor genérico como 1 también funciona.

Siguiendo la receta de diseño, la función para calcular la raíz cuadrada usando el método de Heron queda como sigue:

```

1  # heron_r: num num num -> num
2  # helper funcion de heron, calcula la raiz cuadrada de x
3  # con precision epsilon y valor inicial de estimacion
4  # ejemplo: heron(2,0.00001,1) devuelve 1.414215...
5  def heron_r(x, epsilon, estimacion):
6      # pre-condicion
7      assert x > 0
8      if cerca(estimacion * estimacion, x, epsilon):
9          # caso base
10         return estimacion
11     else:
12         # caso recursivo
13         mejor_estimacion = (estimacion + x / estimacion) / 2
14         return heron_r(x, epsilon, mejor_estimacion)
15
16 # heron: num num -> num
17 # calcula la raiz cuadrada de x, con precision epsilon
18 # ejemplo:
19 # heron(2,0.1) devuelve 1.416...
20 # heron(2,0.00001) devuelve 1.414215...
21 def heron(x, eps):
22     return heron_r(x, eps, x / 2.0)
23
24 import math
25 assert cerca(heron(2, 0.1), math.sqrt(2), 0.1)
26 assert cerca(heron(3, 0.01), math.sqrt(3), 0.01)
27 assert cerca(heron(4, 0.001), math.sqrt(4), 0.001)

```